



Contents lists available at ScienceDirect

Information Processing and Management

journal homepage: www.elsevier.com/locate/ipm

A reliable adaptive prototype-based learning for evolving data streams with limited labels

Salah Ud Din ^{a,b,c}, Aman Ullah ^{a,b}, Cobbinah B. Mawuli ^{a,b}, Qinli Yang ^{a,b}, Junming Shao ^{a,b,*}^a Yangtze Delta Region Institute (Huzhou), University of Electronic Science and Technology of China, Huzhou, 313001, China^b School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, 611731, China^c Department of Computer Science, COMSATS University Islamabad, Abbottabad Campus, Abbottabad, 22020, Pakistan

ARTICLE INFO

Keywords:

Data streams
Data-driven prototypes
Concept drift
Concept evolution
Semi-supervised classification

ABSTRACT

Data stream mining presents notable challenges in the form of concept drift and evolution. Existing learning algorithms, typically designed within a supervised learning framework, require class labels for all data points. However, this is an impractical requirement given the rapid pace of data streams, which often results in label scarcity. Recognizing the realistic necessity of learning from data streams with limited labels, we propose an adaptive, data-driven, prototype-based semi-supervised learning framework specifically tailored to handle evolving data streams. Our method employs a prototype-based data representation, summarizing the continuous flow of streaming data using dynamic prototypes at varying levels of granularity. This technique enables improved data abstraction, capturing the underlying local data distributions more accurately. The model also incorporates reliability modeling and efficient emerging class discovery, dynamically updating the significance of prototypes over time and swiftly adapting to local concept drift. We further leverage these adaptive prototypes to intuitively detect concept evolution, i.e., identifying novel classes from a local density perspective. To minimize the need for manual labeling while optimizing performance, we incorporate active learning into our method. This method employs a dual-criteria approach for data point selection, considering both uncertainty and local density. These manually labeled data points, together with unlabeled data, serve to update the model efficiently and robustly. Empirical validation using several benchmark datasets demonstrates promising performance in comparison to existing state-of-the-art techniques.

1. Introduction

With the constant advancements in technology, a multitude of real-life applications and gadgets are becoming increasingly prevalent, resulting in a substantial volume of data being produced daily across various domains. This data comes from sources such as healthcare, social media, biometric devices, wearable technology, e-commerce, and environmental monitoring. These diverse sources of data present opportunities for exploration, particularly in uncovering patterns and trends. To facilitate this, specialized methods (see Gomes et al. (2017), Gomes, Read, Bifet, Barddal, and Gama (2019), Hasan, Orgun, and Schwitter (2019)) have been developed for efficient and rapid data analysis, aiming to extract valuable insights. One significant task in this data mining process

* Corresponding author at: Yangtze Delta Region Institute (Huzhou), University of Electronic Science and Technology of China, Huzhou, 313001, China.

E-mail addresses: salahuddin@csj.uestc.edu.cn (S.U. Din), dr.aman@csj.uestc.edu.cn (A. Ullah), ben@std.uestc.edu.cn (C.B. Mawuli), qinli.yang@uestc.edu.cn (Q. Yang), junmshao@uestc.edu.cn (J. Shao).<https://doi.org/10.1016/j.ipm.2023.103532>

Received 6 June 2023; Received in revised form 20 September 2023; Accepted 11 October 2023

Available online 25 October 2023

0306-4573/© 2023 Published by Elsevier Ltd.

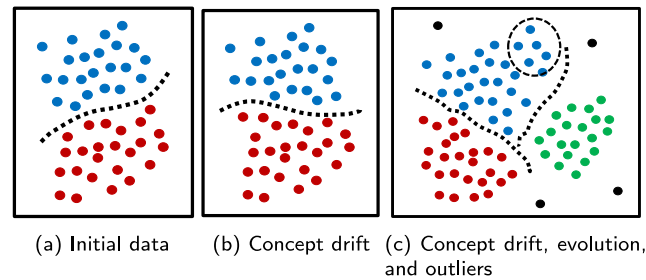


Fig. 1. Learning challenges arise from evolving streaming data due to concept drift, evolution, and outliers. Red and Blue markers represent pre-existing classes, while Green markers indicate a new class. Black markers depict outliers. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

is classification, particularly within the context of streaming data. However, the dynamic nature of such data introduces unique challenges, including concept drift, concept evolution, and issues concerning label accessibility and rarity, distinguishing it from classification within a stationary context (Lu et al., 2019).

Concept drift and evolution. Concept drift occurs when the relationships between input features and target variables in data streams change over time, negatively impacting the performance of machine learning models as they rely on historical data that may not represent the current data distribution (Gama, Žliobaitė, Bifet, Pechenizkiy, & Bouchachia, 2014; Lu et al., 2019). In contrast, concept evolution refers to the appearance of previously unobserved classes or concepts within the streaming data that are not part of the original dataset (Din et al., 2021; Guo, Xia, Li, & Wang, 2023; Parmar, Chouhan, & Raychoudhury, 2023). This phenomenon can pose challenges for learning algorithms that are initially trained on the existing dataset, as they may struggle to accurately classify these new classes. For instance, the ever-changing nature of medical data streams, including patient records and sensor data, presents difficulties in constructing precise models for disease diagnosis or patient monitoring. As health conditions of patients shift or novel diseases arise, the data distribution varies over time, resulting in concept drift in medical diagnosis. Fig. 1 illustrates a toy example of the challenge in distinguishing a newly emerging class amidst concept drift and outliers in evolving streaming data.

Label availability and scarcity. When dealing with streaming data, it becomes essential to persistently update the learning model to handle evolving patterns and changing relationships, which is crucial for maintaining accuracy and effectiveness (Al-Khateeb et al., 2016; Din & Shao, 2020; Guo et al., 2023; ZareMooodi, Kamali Siahroudi, & Beigy, 2019). The majority of prior research on concept drift and evolution has primarily relied on fully supervised methods, assuming instant access to true class labels for immediate model updates (Al-Khateeb et al., 2016; Din & Shao, 2020; Guo et al., 2023; ZareMooodi et al., 2019). However, in high-velocity data streams such as social media or network security, real-time labeling of all data points is impractical and labor-intensive (Fahy, Yang, & Gongora, 2022; Pham, Kottke, Krempel, & Sick, 2022). The assumption that tagging a small amount of data points may be manageable is more plausible in such situations. As a solution to this challenge, researchers have developed semi-supervised methodologies (Din, Kumar, Shao, Mawuli, & Ndiaye, 2022; Din et al., 2020; Ferreira, Zimbrão, & Alvim, 2019; Gu, 2022, 2023; Yin, Chen, & Yin, 2023) that cleverly leverage a combination of a small quantity of labeled data along with a much larger pool of unlabeled data, improving model efficiency and reducing labeling costs. However, their ability to detect new or unseen classes is limited.

Addressing concept drift and evolution while considering label scarcity is crucial, but only a few approaches (Gao, Chandra, Li, Khan, & Bhavani, 2022; Haque, Khan, & Baron, 2016; Liao et al., 2023; Zheng, Li, Hu, & Yu, 2021) have focused on this issue. These strategies employ active learning to mitigate the burden of labeling by selectively seeking true labels for a specific subset of examples, which are then used to update the model, thereby preserving its performance in terms of classification and novel class detection. However, these approaches have potential limitations. They heavily rely on unsupervised techniques based on k-means or graph-based clustering algorithms, which assume accurate representation of classes by hyperspheres, a supposition that may not hold true in various contexts. Additionally, these methods require predefining the number of clusters, which can be challenging in dynamic and complex data environments where the optimal number may not be easily determined. Moreover, techniques operating solely in the unsupervised domain, such as Cai, Zhao, Ting, Mu, and Jiang (2019), Mu, Ting, and Zhou (2017), Mu, Zhu, Du, Lim, and Zhou (2017), Razavi-Far, Hallaji, Saif, and Ditzler (2019), which use unlabeled data for model updates, often struggle to effectively address concept drift.

Addressing these challenges, we develop a novel, adaptive, data-driven prototype-based semi-supervised learning framework designed specifically for evolving data streams. This approach allows for the simultaneous examination of both concept drift and evolution. The key contributions we bring forward include:

- **Prototype-based data representation:** We introduce a prototype-based data abstraction approach that summarizes streaming data by maintaining a dynamic set of prototypes extracted at multiple levels of granularity, offering an intuitive data abstraction that accurately reflects the local data distributions. These prototypes can be incrementally updated in a single pass, enhancing computational efficiency and accurately capturing regions of varying data densities.
- **Reliability modeling and efficient emerging class discovery:** Our model built on adaptive, data-driven prototypes automatically determines the significance of prototypes over time using a disagreement-based method and quickly adapts to local concept drift. It also enables the intuitive detection of emerging classes from a local density perspective.

- **High-performance with limited labels:** In this study, we incorporate active learning to substantially diminish the need for manual labeling, while simultaneously preserving or even enhancing the performance of our model. Validation of this approach on several benchmark datasets demonstrated superior prediction performance compared to other techniques while using only a limited number of labels.

The rest of this paper is structured as follows: Section 2 delves into the relevant literature. Section 3 introduces the proposed method in detail. Detailed descriptions of the conducted experiments along with an in-depth discussion of their outcomes can be found in Section 4. The paper is wrapped up in Section 5, providing the final thoughts and conclusions on the work.

2. Related work

The field of data stream learning has witnessed the emergence of numerous innovative techniques in recent years. However, given the limitations on space, this section provide a concise overview, focusing primarily on the methods that have been most relevant in the existing literature.

Concept drift adaptation. Concept drift is a key challenge in the data streams where data at different time intervals – marked as t and $t + \Delta$ – may stem from distinct joint probability distributions, denoted as $P_t(x, y)$ and $P(t + \Delta)(x, y)$. This joint probability can be segmented into, which represents the distribution of inputs $P_t(x)$, and $P_t(y|x)$, the conditional distribution of outputs given those inputs. The latter defines the true classification rules for the problem. Concept drift can manifest either through shifts in the input distribution, known as “virtual drifts”; through alterations in the conditional output distribution, termed “real drifts”; or through simultaneous changes in both. It is important to note that only real drifts change the inherent decision criteria of the problem. However, the performance of the decision criteria generated by the model can be affected by both real and virtual drifts, and the extent of this impact varies based on the learning algorithm and model being employed.

Adaptive algorithms are crucial in mitigating the issue of concept drift by continuously monitoring and adjusting the model to align with the evolving data landscape, thereby maintaining high performance levels. Various methodologies have been employed to tackle this, and they can generally be grouped into two main categories: explicit change detection methods and implicit techniques. Change detection methods (Haque et al., 2016; Komorniczak, Zyblewski, & Ksieniewicz, 2022) scrutinize the performance metrics of a model on new, incoming data. Statistical tests are employed to assess whether there has been a significant shift in the data distribution. Upon detecting such a change, one has two options: either rebuild the model from scratch or selectively update the segments affected by the drift (Shao, Huang, Yang, & Luo, 2018). Implicit techniques (e.g., Losing, Hammer, and Wersing (2017)) take a different approach. Rather than actively searching for changes, these techniques gradually integrate new concepts by making incremental adjustments to the learning process. Broadly speaking, algorithms for learning from data streams can be classified into single learning models and ensemble methods. In single learning models, a single predictive model is constantly updated with new data (Din et al., 2022). On the other hand, ensemble methods (e.g. Cano and Krawczyk (2020, 2022), Krawczyk and Cano (2019)) maintain a collection of models, each trained on varying data distributions. These methods dynamically switch between these models according to the current data distribution, and the performance of all ensemble members is regularly evaluated. The least effective member is then replaced by a new, more accurate classifier. The majority of research in this area has been conducted using supervised learning methodologies, where accurate labels for all data points are readily available for model training and adjustment. However, obtaining complete and accurate labels for all data points is often impractical in real-world scenarios. To address this issue, semi-supervised learning techniques have been proposed. These methods enable the model to learn from both labeled and unlabeled data, thereby reducing the cost and effort required for labeling.

Semi-Supervised Learning Methods. In an effort to minimize dependency on labels, a variety of static semi-supervised learning (SSL) algorithms, including self-training, co-training, graph-based and clustering methods, have been examined and adapted to function effectively with data stream and manage concept drift.

Graph-based methods, which operate based on the manifold assumption, have been notably utilized for streaming data. For instance, Wagner, Guha, Kasiviswanathan, and Mishra (2018) propose a temporal label propagation approach specifically designed for data stream, whereas the algorithm presented in Zhu, Pang, Sarrafzadeh, Ban, and Inoue (2016) employs an online max-flow method to classify unlabeled data points. Co-training-based methods have also been explored, with Wang et al. (Wang & Li, 2018) proposing the SCo-forest algorithm designed specifically for streaming settings, and the self-training strategies proposed in Khezri, Tanha, Ahmadi, and Sharifi (2020) incrementally updating the classifier using predicted labels. Clustering-based methods have gained popularity in SSL due to the inherent assumption that data points within the same cluster are likely to share class labels. Methods like SPASC (Hosseini, Gholipour, & Beigy, 2016) have leveraged this premise to infer labels for other data points within a cluster.

In complement to the methods mentioned, a number of studies have employed active learning strategies (Aguiar & Cano, 2023; Korycki, Cano, & Krawczyk, 2019; Kranjc et al., 2015; Mohamad, Sayed Mouchaweh, & Bouchachia, 2018) as a means to curtail the substantial labeling expenses that often accompany streaming data. These methods operate by selecting specific data points from the streaming data, guided by certain heuristics, and then requesting users to supply labels for these chosen data points. Once these labeled instances are received, the model is updated accordingly. By engaging with active learning techniques, these studies highlight a viable pathway for reducing labeling costs while ensuring the ongoing adaptability and effectiveness of the model.

Novel Class Discovery. The algorithms for detecting novel classes primarily fall into two categories: model-based techniques and clustering-based strategies. Clustering-based algorithms (Al-Khateeb et al., 2016; Din & Shao, 2020; Gao et al., 2022; Liao et al., 2023; ZareMoodi et al., 2019; Zheng et al., 2021), employ clustering tactics like k-means to define decision boundaries capable of

Table 1
Summary of variables, notations and symbols.

Symbol	Description
S	A continuous flow of data points, denoted as $S = \{x_t, y_t\}_{t=1}^{\infty}$
S_{init}	The data points used for the initial training
$\mathcal{K}$	Block size
S_i	A block of data points, i.e., $S_i = \{x_1, x_2, \dots, x_{\mathcal{K}}\}$
x, z	Data points
$\mathcal{Y}$	A set of classes denoted by $\mathcal{Y} = \{1, 2, \dots, C\}$.
y	An actual label of the class $y \in \mathcal{Y}$ that x belongs to
$\hat{y}$	The predicted label by the model
C	Total number of classes
c	A class labeled as c
$\mathcal{X}$, and $\mathcal{X}^c$	$\mathcal{X}$ denotes a set of data points, and $\mathcal{X}^c$ represents the set of data points belonging to class c
μ , and μ^c	μ is the mean of the set $\mathcal{X}$, and μ^c is the mean of the set $\mathcal{X}^c$
$\mathcal{E}$, and $\mathcal{E}^c$	$\mathcal{E}$ is the mean scalar product of the set $\mathcal{X}$, and $\mathcal{E}^c$ is the mean scalar product of the set $\mathcal{X}^c$
N , and N^c	N is the total number of data points in set $\mathcal{X}$, and N^c is the total number of data points in the set $\mathcal{X}^c$
LS^c , and SP^c	LS^c is the linear sum of all data points in the set $\mathcal{X}^c$, while SP^c represents the sum of the scalar product of all data points in the set $\mathcal{X}^c$
Π^c	A 3-tuple represented as $\Pi^c = (LS^c, SP^c, N^c)$
PT	A 5-tuple represented as PT (prototype) = $(LS^p, SP^p, N^p, \Omega, Y)$
Ω	The weight/reliability assigned to each prototype
Y	A variable used to store time
$\mathcal{L}$	Maximum levels (layers)
PT_{ℓ}^c	A set of prototypes for class c at a particular level of granularity ($\ell = 1, 2, \dots, \mathcal{L}$)
Γ_{ℓ}^c	Γ_{ℓ}^c represents the radius of the local influential area around each prototype, corresponding to the ℓ^{th} level of granularity
δ	Rate of decay
β	The size of the buffer

distinguishing new classes. They delineate clusters in the feature space, each marked by a distinct radius and center, and categorize data points within these clusters as known or regular concepts. Data points positioned outside these clusters are identified as potential outliers or emerging classes and are preserved in the buffer for further scrutiny. Notably, methods presented in [Din and Shao \(2020\)](#), [Liao et al. \(2023\)](#) use the Local Outlier Factor (LOF) algorithm along with a local density method for novel class detection. In contrast, techniques such as [Al-Khateeb et al. \(2016\)](#), [Gao et al. \(2022\)](#), [Zheng et al. \(2021\)](#) necessitate additional identification to confirm outliers as new classes, often by analyzing the density of clusters based on intra-class and inter-class relationships among data points.

In contrast, model-based learning methods ([Bouguelia, Nowaczyk, & Payberah, 2018](#); [Mu, Ting, & Zhou, 2017](#); [Mu, Zhu, et al., 2017](#)) aim to uncover predictive models that can identify emerging classes. These methods employ various learning models, including sketch-based, graph-based, and decision tree-based algorithms, to learn underlying patterns and trends in the data. Instead of solely clustering data points, model-based techniques use learned models to predict and classify new data points as they emerge. Methods involving matrix sketches, as presented in [Mu, Zhu, et al. \(2017\)](#), condense global and local information into low-dimensional matrices. These matrices aid in detecting novel class data points based on their distance from frequent directions within the global sketches. Graph-based algorithms, like the one described in [Bouguelia et al. \(2018\)](#), utilize a dynamic graph structure where data points are represented as nodes connected by edges. This structure adapts to include new data, and data points falling outside established hyperspheres are identified as potential new classes. The approach in [Mu, Ting, and Zhou \(2017\)](#) uses a model based on random trees, dividing the data space into normal and outlier regions based on the path length of training samples within the tree structure.

3. Proposed method

This section aims to provide a thorough explanation of the method proposed in the paper. We first provide a summary of the key terms and notations used in this paper. [Table 1](#) contains a list of these variables, notations, and symbols, which are frequently referenced throughout the paper. Afterward, a general outline of the algorithm is presented, followed by a more detailed description.

3.1. Notations, symbols and definitions

A continuous flow of data points, denoted as $S = \{(x_t, y_t)\}_{t=1}^{\infty}$, characterizes a data stream, with each point (x_t, y_t) consisting of a d -dimensional feature vector and an associated actual label $y_t \in \mathcal{Y}$ (or $y_t = \phi$ if unavailable) at time step t . The data stream is partitioned into segments of fixed length, known as data blocks, which accumulate incoming data points before algorithmic processing. These data blocks are represented as $S = \{S_1, S_2, \dots, S_i, \dots, S_k, \dots\}$, with S_i signifying the i -th data block in the stream and S_k as the latest block. The number of data points within each block is defined by the block size, $\mathcal{K}$, i.e., $S_i = \{x_1, x_2, \dots, x_{\mathcal{K}}\}$.

Definition 1 (*Classification with Novel Class Discovery*). Let S_{init} represent a training dataset containing m samples from a data stream, specifically, $\mathcal{X} = S_{init} = \{(x_i, y_i)\}_{i=1}^m$, where each $x_i \in \mathcal{R}^d$ and the corresponding $y_i \in \mathcal{Y} = \{1, 2, \dots, C\}$ denotes the class label. The classes within the set $\mathcal{Y}$ are referred to as known (existing) labels or classes. Initially, a learning model $f : \mathcal{X} \rightarrow \mathcal{Y}$ is constructed using the training set S_{init} . The learned model is responsible for three main tasks: (1) discovering emerging classes, (2) classifying data points within known classes, and (3) updating the model to incorporate new data and newly discovered classes (Mu, Ting, & Zhou, 2017). This enables the model to provide accurate predictions even in the presence of concept drift and evolution. As new data points from the streaming data arrive, the current decision model performs a filtration process to distinguish whether the data points likely belong to known classes or represent a new type of data (new emerging class). If the data points are presumed to belong to known classes, their class labels are predicted by the model. However, if the data points are flagged as probable novel class data, it means they do not belong to any existing class. Consequently, if a novel class is discovered, it is integrated into the model by expanding the label set $\mathcal{Y} = \{1, 2, \dots, C, C+1\}$, allowing for accurate classification of future data points belonging to the newly identified class.

Definition 2 (*Local Density*). Local density (LD) is a concept within the recently developed Empirical Data Analysis (EDA) framework (Angelov, Gu, & Príncipe, 2018; Gu, Angelov, & Príncipe, 2018), which is designed to analyze and summarize data to understand their primary characteristics and reveal hidden patterns, trends, or relationships. EDA measures, such as local density, are aimed at extracting ensemble properties from observed data without requiring any prior assumptions. Within the EDA framework, local density serves as a measure for identifying the primary local peaks (maxima) of data distribution. Derived empirically from all observed data points, LD is computed in a parameter-free manner. The local density of a data point x is expressed as follows:

$$LD(x_i) = \frac{1}{1 + \frac{\|x_i - \mu\|^2}{\mathcal{E} - \|\mu\|^2}} \quad (1)$$

where x_i is a specific data point from the data ($\mathcal{X}$), μ is the mean, representing the average of all data points (i.e., $\mu = \frac{1}{N} \sum_{i=1}^N x_i$), $\mathcal{E}$ is the mean of scalar products of all data points (i.e., $\mathcal{E} = \frac{1}{N} \sum_{i=1}^N \|x_i\|^2$ where $\|x_i\| = \sqrt{x_i^T x_i}$), and N is the total number of data points. The LD measure helps uncover the underlying structure of data without depending on assumptions about the data generation process or specific parameters. In essence, it helps identify local regions where data points are most densely concentrated.

3.2. Method overview

In Section 1, we introduced our method, which aims to identify emerging classes and reduce dependence on the complete label set, which is impractical in streaming data. Our method combines various techniques to efficiently process concept-drifting streaming data using both limited labeled data and plentiful unlabeled data. The proposed technique starts with the initialization of the learning model using initial training data (cf. Section 3.3.1). For each class present in the training data, a set of prototypes at different levels of granularity is extracted using a data-driven approach, which maintains statistical summaries. When new data arrives, we determine whether each data point falls within or outside the boundaries defined by the radius of each prototype. All data points that are within the boundaries are classified by the algorithm (cf. Section 3.3.2). Conversely, data points outside the boundary are deemed probable new class data points and are stored in a buffer. The emerging class discovery component (cf. Section 3.3.3) then examines the buffer to declare an emerging class if novel class data points are detected. However, if the data points are determined to be outliers of known classes, the algorithm proceeds to classify them. To stay current with the evolving concepts in the data stream and accommodate concept drift and evolution, the model is persistently updated with the most recent partially labeled and unlabeled data blocks (cf. Section 3.3.4). The update procedure integrates active learning (for minimizing reliance on labeled data), disagreement-based learning, consideration of time impact, and an online learning mechanism. An overview of the proposed method is illustrated in Fig. 2, while the following sections delve into the specifics of each component.

3.3. Main components of the proposed method

The proposed method consists of four primary components: learning model initialization, classification, novel class discovery and online learning mechanism. Detailed explanations of these components are provided in the subsequent sections.

3.3.1. Learning model initialization

The initial step of our proposed technique involves invoking a function to initialize the learning model. This function is defined in Algorithm 1 and requires S_{init} (initial training data) and $\mathcal{L}$ (max. number of layers) as input parameters. It returns a collection of prototypes along with other global meta parameter as output. First, the data in S_{init} is partitioned into distinct subsets ($\{\mathcal{X}^c\}_{c=1}^C = \{\mathcal{X}^1, \mathcal{X}^2, \dots, \mathcal{X}^C\}$) such that each subset contains samples of a single class ($\mathcal{X}^c$). For each class, we maintain a local meta parameter (Π^c) in the form of a 3-tuple, represented as $\Pi^c = (LS^c, SP^c, N^c)$. In this representation, LS^c refers to the linear sum, which is the sum of all samples in the class, calculated as $LS^c = \sum_{i=1}^{N^c} x_i$, where $x_i \in \mathcal{X}^c$. SP^c is the sum of the scalar products of all samples, computed as $SP^c = \sum_{i=1}^{N^c} \|x_i\|^2$, where $\|x_i\| = \sqrt{x_i^T x_i}$. N^c denotes the total number of data samples in the class. Due to the incremental and additive properties of the local meta parameter, new data points can be added or removed incrementally during online learning from streaming data. This is done by updating the local meta parameter using the following equations:

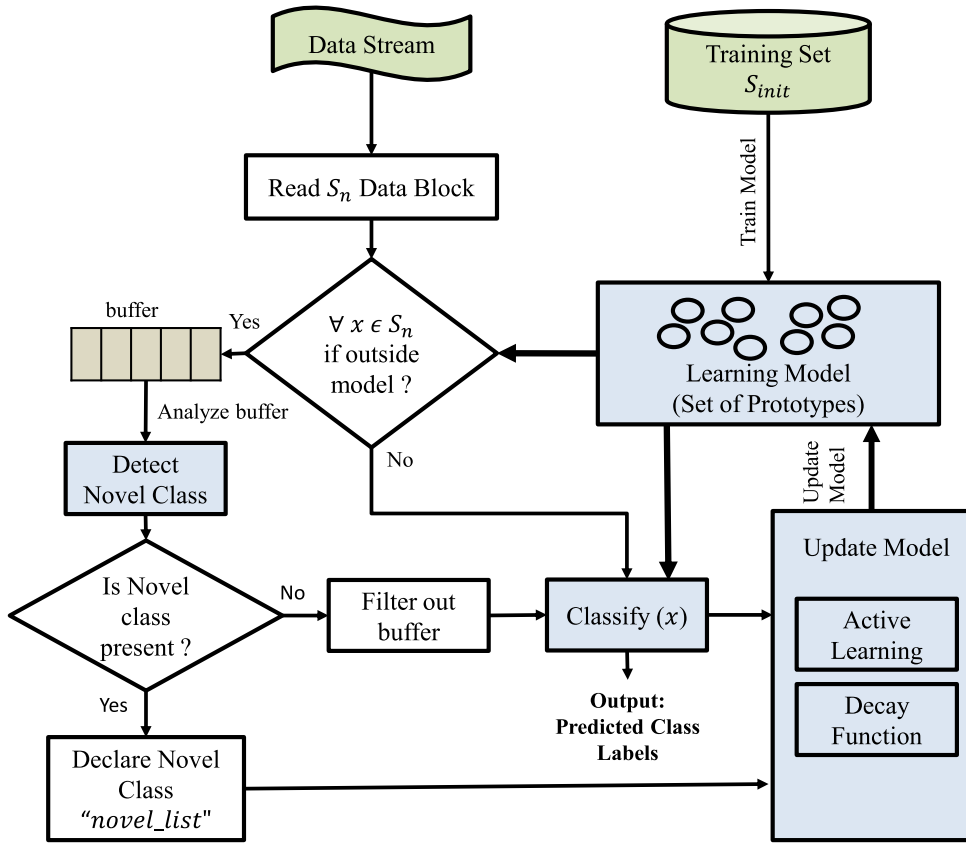


Fig. 2. Overview of adaptive data-driven prototype-based learning for evolving data streams.

($LS^c = LS^c \pm x^c$, $SP^c = SP^c \pm \|x^c\|^2$, $N^c = N^c \pm 1$). The local meta parameter's desirable properties enable us to calculate the class mean, $\mu^c = \frac{LS^c}{N^c}$, and the mean scalar product, $\mathcal{E}^c = \frac{SP^c}{N^c}$.

For each class, a set of prototypes ($\{PT_\ell^c\}_{\ell=1}^L$) at different levels of granularity (hierarchical structure) is extracted using a data-driven approach. Each prototype (PT) maintains a statistical summary in the form of a 5-tuple, represented as $PT = (LS^p, SP^p, N^p, \Omega, Y)$. In this representation, LS^p refers to the linear sum, SP^p is the scalar product of all samples belonging to the prototype, and N^p denotes the total number of samples associated with the prototype. Ω is the weight of the prototype, initially set to 1, and Y indicates the last update time for the prototype. By maintaining these statistical summaries in the form of prototypes, the learning model can effectively represent different levels of granularity within the data. Higher-level prototypes represent a more general view of the data, while lower levels provide more detailed representations of the data.

The procedure for extracting prototypes for a specific class c is explained as follows.

Step 1. Local density calculation: For each data sample $x_i \in \mathcal{X}^c$, a local density (LD) measure is calculate using Eq. (1). Local density identifies the primary local maxima of the data distribution.

Step 2. Ranking local density: This step is used to determine a ranking order for the data samples based on their local density values. Once the local density values have been computed for all data samples, the one with the highest value denoted as r^1 is selected, i.e., (global maximum) $r^1 = \arg\max_{i=1,2,\dots,N^c} LD(x_i)$. This value is added to an indexing list called $\{r_{list}\}$ as the first element. If there are multiple maxima, any one can be chosen to start with. The rest of the elements in $\{r_{list}\}$ are found one by one according to the following method. A data sample closest to r^1 is identified as r^2 (i.e., $r^2 = \arg\min_{i=1,2,\dots,N^c; i \notin \{r_{list}\}} \|x_{r^1} - x_i\|$), added to $\{r_{list}\}$. r^2 becomes the new global maximum denoted as $r^m = r^2$. This process is repeated, with the next data sample closest to r^m added to $\{r_{list}\}$, and set as the new global maximum. The ranking operation continues until all data samples have been processed, resulting in a ranked list of data samples, denoted as $\{r_{list}\} = \{r^i | i = 1, 2, \dots, N^c\}$, and their corresponding local density values as $LD(\{r_{list}\}) = \{LD(r^1), LD(r^2), \dots, LD(r^{N^c})\}$.

Step 3. Local maxima identification and data clouds formation: Using the values in $\{r_{list}\}$, we can identify the local maxima of local density by following this equation (for $i = 2, 3, \dots, N^c - 1$):

$$\begin{aligned} &\text{IF } (LD(r^{i-1}) < LD(r^i) \text{ and } LD(r^i) > LD(r^{i+1})) \\ &\text{THEN } (\{r_{max}\} = \{r_{max}\} \cup r^i) \end{aligned} \quad (2)$$

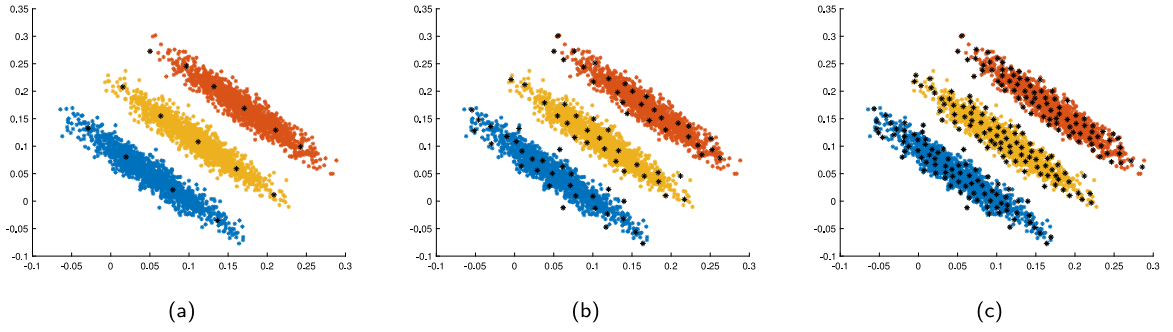


Fig. 3. Extracted prototypes (black) at different levels of granularity (a) 1th level, (b) 3rd level, and (c) 5th level.

Here, $\{r_{max}\}$ represents the set of local maxima of local density that are identified based on the values in $\{r_{list}\}$. We add r^i to the set if its local density value is greater than its adjacent values in $\{r_{list}\}$.

Afterward, data points ($x_i \in \mathcal{X}^c$) corresponding to the local maxima $\{r_{max}\}$ list are used to create Voronoi tessellations in the data space, attracting nearby data samples to form a number of clusters denoted as $\{\mathcal{H}_i\}_{i=1}^n$, where n is the total number of clusters. The density values at the centers of the clusters, $\{\mathcal{H}_i\}_{i=1}^n$, are then calculated using the following equation for $i = 1, 2, \dots, n$:

$$LD(h_i) = \frac{\eta_i}{1 + \frac{\|h_i - \mu^c\|^2}{\mathcal{E}^c - \|\mu^c\|^2}} \quad (3)$$

Here, η_i and h_i represent the cardinality and center ($h_i = \frac{1}{\eta_i} \sum_{x \in \mathcal{H}_i} x$) of $\mathcal{H}_i$, respectively.

Step 4. Prototype extraction: In this stage, first, we calculate the radius of the local influential area around each prototype, which represents the average distance between two closely connected prototypes at a particular level of hierarchy. This is based on the distances between observed data samples and the level of detail chosen by the user. We begin by calculating the radius, denoted as Γ_1^c , at the first level of hierarchy ($\ell = 1$) using the formula:

$$\Gamma_1^c = \frac{1}{\hat{N}_1^c} \sum_{x, z \in \mathcal{X}^c; x \neq z; \|x - z\|^2 \leq \Gamma_0^c} \|x - z\|^2 \quad (4)$$

In this equation, $\hat{N}_1^c$ represents the count of paired data points where the distance between the pair is less than the average squared distance between any two data points from the c th class. The average squared distance is calculated as $\Gamma_0^c = 2(\mathcal{E}^c - \|\mu^c\|^2)$.

For higher granularity levels, specifically for $\ell = 2, 3, \dots, \mathcal{L}$, the radius can be determined in a sequential manner utilizing the given equation:

$$\Gamma_\ell^c = \frac{1}{\hat{N}_\ell^c} \sum_{x, z \in \mathcal{X}^c; x \neq z; \|x - z\|^2 \leq \Gamma_{\ell-1}^c} \|x - z\|^2 \quad (5)$$

The variables Γ_ℓ^c and $\Gamma_{\ell-1}^c$ indicate the boundary of the local influential areas for the ℓ^{th} and $(\ell-1)^{th}$ granularity levels, respectively. The value of $\hat{N}_\ell^c$ represents the count of data point pairs whose separation is less than $\Gamma_{\ell-1}^c$.

In order to pinpoint the most representative prototypes across various hierarchical levels, we utilize a filtering method that takes into account both multi-modal density values and distances to prevent less representative local peaks from being included. To extract prototypes at the ℓ^{th} level of hierarchy, we initially need to determine the collection of neighboring clusters (denoted by $nbr(h_i)$) for each cluster h_i . This is done by using the following formula, which is reliant on the mutual distances between cluster centers ($i, j = 1, 2, \dots, n; i \neq j$):

$$nbr(h_i) = \{h_j \in \{\mathcal{H}\}_1^n \mid dist^2(h_i, h_j) \leq \Gamma_\ell^c\} \quad (6)$$

Here, Γ_ℓ^c represents the mean of local influence zone (radius) surrounding each data point, in relation to the ℓ^{th} ($\ell = 1, 2, \dots, \mathcal{L}$) level of hierarchy and it is derived from the data of the c th class according to user preferences. Subsequently, we pick the most representative cluster centers of the c th class, expressed as $\{h_i^*\}_{i=1}^n$, from the existing data centers ($\{h_i\}_{i=1}^n$) that satisfy the equation:

$$\begin{aligned} &\text{IF } (LD(h_i) > \max_{h \in nbr(h_i)} (LD(\hat{h}))) \\ &\text{THEN } (h^* = h^* \cup h_i) \end{aligned} \quad (7)$$

After selecting the set of most representative centers $\{h_i^*\}_{i=1}^n$, we use them to create Voronoi tessellations in the data space. This attracts nearby data samples and forms new clusters denoted as $\{\mathcal{G}_i\}_{i=1}^n$. For each newly formed cluster $\mathcal{G}_i$, we compute a statistical summary in the form of a prototype, denoted as $PT = (LS^p, SP^p, N^p, \Omega, Y)$, where $LS^p = \sum_{i=1}^{N^p} x_i$, and $SP^p = \sum_{i=1}^{N^p} \|x_i\|^2$ for all $x_i \in \mathcal{G}_i$. We then save and add each prototype to the prototype collection $PT_\ell^c = PT_\ell^c \cup PT$.

Algorithm 1: Learning Model Initialization (Prototype Extraction)

```

1 Function ModelInit( $S_{init}, \mathcal{L}$ )
2    $Model \leftarrow \phi$ 
3    $GlobalMeta \leftarrow \phi$ 
4    $\{\mathcal{X}^c\}_{c=1}^C \leftarrow \text{PARTITION}(S_{init})$  // Partition  $S_{init}$  into distinct classes
5   foreach  $\mathcal{X}^c \in \{\mathcal{X}^c\}_{c=1}^C$  do
6      $N^c \leftarrow |\mathcal{X}^c|$  // total class samples
7      $LS^c \leftarrow \sum_{j=1}^{N^c} x_j$  // Linear sum
8      $SP^c \leftarrow \sum_{j=1}^{N^c} \|x_j\|^2$  // Scalar product of class samples. Here  $\|x\| = \sqrt{x^T x}$ 
9      $\Pi^c \leftarrow (LS^c, SP^c, N^c)$ 
10    forall  $x_i \in \mathcal{X}^c$  compute local density ( $LD(x_i)$ ) using Eq. (1)
11    Compute ranking order  $\{r_{list}\}$  for all data samples based on  $LD(x_i)$  // see section 3.3.1
12    Extract local maxima  $\{r_{max}\}$  based on  $\{r_{list}\}$  using Equation (2)
13    Create data clouds (clusters  $\{\mathcal{H}_i\}_{i=1}^n$ ) using local maxima  $\{r_{max}\}$ 
14    For all cluster centers  $h_i \in \{\mathcal{H}_i\}_{i=1}^n$  compute local density  $LD(h_i)$  using Eq. (3)
15     $\ell \leftarrow 1$ 
16    while  $\ell \leq \mathcal{L}$  do
17      Compute influence area  $\Gamma_\ell^c$  using Eq. (4) and (5)
18      Extract most representative centers  $\{h_i^*\}_{i=1}^{n^*}$  from  $\{h_i\}_{i=1}^n$  using Eq. (6) and (7)
19       $\{\mathcal{G}_i\}_{i=1}^{n^*} \leftarrow \text{CREATEDATA CLOUDS}(S_{init}, \{h_i^*\}_{i=1}^{n^*})$  // Create new clusters in data space ( $S_{init}$ )
20      foreach  $\mathcal{G}_i \in \{\mathcal{G}_i\}_{i=1}^{n^*}$  do
21        Create a  $PT = (LS^p, SP^p, N^p, \Omega, Y)$ , where  $LS^p \leftarrow \sum_{i=1}^{N^p} x_i$ ,  $SP^p \leftarrow \sum_{i=1}^{N^p} \|x_i\|^2$ , ( $\forall x_i \in \mathcal{G}_i$ ),  $\Omega \leftarrow 1, Y \leftarrow 1$ .
22         $PT_\ell^c = PT_\ell^c \cup PT$ 
23      end
24       $\ell \leftarrow \ell + 1$ 
25    end
26     $meta^c \leftarrow (\Pi^c, \{\Gamma_\ell^c\}_{\ell=1}^{\mathcal{L}})$ 
27     $GlobalMeta \leftarrow GlobalMeta \cup meta^c$ 
28     $Model \leftarrow Model \cup \{PT_\ell^c\}_{\ell=1}^{\mathcal{L}}$ 
29  end
30  return  $Model, GlobalMeta$ 
31 End Function

```

It is important to highlight that Eq. (7) adeptly eliminates less representative prototypes by comparing them with their neighboring values, considering their multimodal density. Both Eqs. (6) and (7) greatly streamline the procedure of searching for the most representative data samples within the data space. These prominent prototypes accurately echo the collective characteristics and shared distribution of the data, since the identification procedure takes into account only the multimodal density values and mutual distances of these observed data points.

Furthermore, we maintain a local meta parameter $meta^c$ for every class, which contains the parameters Π^c and Γ_ℓ^c , i.e., $meta^c = (\Pi^c, \{\Gamma_\ell^c\}_{\ell=1}^{\mathcal{L}})$. These parameters will be updated during the online learning process. Additionally, prototypes extracted for every class are stored as $Model = (\{PT_\ell^1\}_{\ell=1}^{\mathcal{L}}, \{PT_\ell^2\}_{\ell=1}^{\mathcal{L}}, \dots, \{PT_\ell^C\}_{\ell=1}^{\mathcal{L}})$. Moreover, we use $GlobalMeta$ to save the local meta parameter for every class, i.e., $GlobalMeta = (meta^1, meta^2, \dots, meta^C)$. Finally, Fig. 3 displays the extracted prototypes, depicted in black, at different levels of granularity.

3.3.2. Classification

In Algorithm 4, we employ a two-step procedure to predict the class label of a new data point within data streams. Firstly, in line 5, we determine if the test instance x is an outlier. Provided x is not an outlier, we promptly classify it using the “nearest prototype” principle, as outlined in Eq. (8). Here, we identify the class label c that yields the smallest Euclidean distance between x and the prototype PT_ℓ^c .

$$plabel = \underset{c=1,2,\dots,C}{\operatorname{argmin}} \|x - PT_\ell^c\| \quad (8)$$

It is important to note that a novel class data point is anticipated to be an outlier. However, not all outliers are necessarily members of a novel class. Consequently, we conduct further analysis on the outliers to ascertain if they genuinely belong to a new class.

Algorithm 2: Novel Class Discovery

```

1 Function ClassDisc( $B, Model, GlobalMeta, \mathcal{L}$ )
2    $NvList \leftarrow \phi$  // novel class list
3    $ExList \leftarrow \phi$  // existing class list
4    $PT_{\mathcal{L}}^E \leftarrow Model.\{PT_{\mathcal{L}}^c\}_{c=1}^C$  // Existing last level prototypes
5    $\{\Gamma_{\mathcal{L}}^{avg}\}_{\mathcal{L}=1}^{\mathcal{L}} \leftarrow AVERAGE(meta^c.\{\Gamma_{\mathcal{L}}^c\}_{\mathcal{L}=1}^{\mathcal{L}})$  // Average radius at all levels
6    $\{\mathcal{G}_i\}_{i=1}^n \leftarrow CREATEDATA CLOUDS(B, \Gamma_1^{avg})$  // Cluster buffer  $B$  using first level average radius. Section 3.3.1
7    $\hat{\mathcal{G}}_i \leftarrow \text{argmax}_i(|\mathcal{G}_i| \in \{\mathcal{G}_i\}_{i=1}^n)$ 
8   if  $|\hat{\mathcal{G}}_i| > (0.20 \times |B|)$  then
9     // Clusters are less representative, if they contain less than 20% data
10    if  $|\hat{\mathcal{G}}_i| > (0.80 \times |B|)$  then
11       $NvList \leftarrow \hat{\mathcal{G}}_i$  // Cluster is considered as novel if it contains 80% of data
12       $\{\mathcal{G}_i\}_{i=1}^n \leftarrow \{\mathcal{G}_i\}_{i=1}^n - \hat{\mathcal{G}}_i$ 
13    end
14    Extract last level prototypes  $PT_{\mathcal{L}}^B$  for buffer ( $B$ ) using  $\Gamma_{\mathcal{L}}^{avg}$  // see Section 3.3.1
15    Assign all  $x_i \in \{\mathcal{G}_i\}_{i=1}^n$  to nearest  $PT \in PT_{\mathcal{L}}^B$ 
16    foreach  $PT \in PT_{\mathcal{L}}^B$  |  $PT$  has  $x_i$  do
17       $\{PT_o\}_{o=1}^k \leftarrow \text{FINDNEARESTPT}(PT, PT_{\mathcal{L}}^B)$  // Find  $k$  nearest neighbor of  $PT$  in  $PT_{\mathcal{L}}^B$ 
18       $\{PT_e\}_{e=1}^k \leftarrow \text{FINDNEARESTPT}(PT, PT_{\mathcal{L}}^E)$  // Find  $k=3$  nearest neighbor of  $PT$  in  $\{PT_{\mathcal{L}}^c\}_{c=1}^C$ 
19       $d^B \leftarrow \text{mean}(\text{dist}(\{PT_o\}_{o=1}^k, PT))$  and  $d^E \leftarrow \text{mean}(\text{dist}(\{PT_e\}_{e=1}^k, PT))$ 
20       $P(\text{Novel}|PT) \leftarrow \frac{d^E}{d^B + d^E}$  and  $P(\text{Exigting}|PT) \leftarrow \frac{d^B}{d^B + d^E}$ 
21      if  $P(\text{Novel}|PT) \geq P(\text{Exigting}|PT)$  then
22         $NvList \leftarrow NvList \cup \{\forall x \in B \mid x \in PT\}$ 
23      else
24         $ExList \leftarrow ExList \cup \{\forall x \in B \mid x \in PT\}$ 
25      end
26    end
27  end
28  if  $|NvList| \geq \frac{|B|}{2}$  then // see Section 3.3.1 Steps 1–4
29     $\{PT_{\mathcal{L}}^{nv}\}_{\mathcal{L}=1}^{\mathcal{L}} \leftarrow \text{EXTRACTPROTOTYPES}(NvList)$ 
30    Compute  $meta^{nv} \leftarrow (\Pi^{nv}, \{\Gamma_{\mathcal{L}}^{nv}\}_{\mathcal{L}=1}^{\mathcal{L}})$ 
31     $GlobalMeta \leftarrow GlobalMeta \cup meta^{nv}$ 
32     $Model \leftarrow Model \cup \{PT_{\mathcal{L}}^{nv}\}_{\mathcal{L}=1}^{\mathcal{L}}$ 
33     $C \leftarrow C + 1$ 
34     $B \leftarrow \phi$ 
35  else
36     $B \leftarrow B - ExList$ 
37     $\{\hat{x}\} \leftarrow \text{FILTEROUT}(B \mid \text{age}(x) \in B > 2 \times \mathcal{K})$  // Remove all data points in  $B$  whose age is greater than  $2 \times \mathcal{K}$  (block size)
38     $ExList \leftarrow ExList \cup \{\hat{x}\}$ 
39     $NvList \leftarrow \phi$ 
40  end
41  return  $NvList, ExList$ 
42 End Function

```

Definition 3 (Outlier). An outlier is a test data point that lies beyond the influence area, or radius, of all prototypes across each granularity level. Such a data point does not closely relate to any existing prototypes, making it distinct and separate from the established groups or classes. This concept is helpful for identifying new or unusual patterns in the data.

3.3.3. Novel class discovery

The concept of discovering emerging classes (as discussed in Al-Khateeb et al. (2016), ZareMoodi et al. (2019)) is based on the notion that each class should have a unique characteristic. Cohesion among data points within the same class and separation from data points in other classes are crucial aspects. Upon introducing new data, the first step is to identify any outliers that are separated from the existing class data (as defined in Definition 3). Subsequently, a filtering process eliminates outliers potentially caused by concept drift. If the remaining outliers exhibit strong cohesion, it validates the existence of an emerging class.

In order to discover novel classes, outlier data points are temporarily stored in a buffer denoted as B . When the buffer's size reaches a certain threshold (β), the data points within it undergo further examination to discern whether they were mistakenly labeled as outliers due to noise, a decision boundary shift caused by concept drift, or inadequate training data. To accomplish this, we start by calculating the average first-level radius using the existing class radii ($\{\Gamma_{\mathcal{L}}^c\}_{c=1}^C$). Following this, we cluster all the data points in the buffer using the steps defined in Section 3.3.1 (Steps 1–4). If the maximum count (i.e., the number of data points assigned to a cluster) is less than 20% of the buffer size, the clusters are considered to be less representative, and the buffer is

filtered out to remove old data points. On the other hand, if the cluster size is greater than 80% of the buffer size, the cluster is considered as comprising novel class data points. Subsequently, we extract the last-level prototypes PT_{ℓ}^B for the buffer (B) based on the average radius, and then assign all data points in the remaining clusters (or all clusters) to the nearest PT . For each prototype $PT \in PT_{\ell}^B$, we identify k nearest neighbor prototypes, $\{PT_o\}_{o=1}^k$ from the outlier prototypes (PT_{ℓ}^B) and $\{PT_e\}_{e=1}^k$ from the existing last-level prototypes ($\{PT_{\ell}^c\}_{c=1}^C$). Using these prototypes, we compute a probability score for each PT using the following formulas:

$$P(Novel|PT) = \frac{d^E}{d^B + d^E} \quad (9)$$

$$P(Existing|PT) = \frac{d^B}{d^B + d^E} \quad (10)$$

In these equations, d^B and d^E represent the mean distances from $\{PT_o\}_{o=1}^k$ and $\{PT_e\}_{e=1}^k$ to PT , respectively (i.e., $d^B = \text{mean}(\text{dist}(\{PT_o\}_{o=1}^k, PT))$ and $d^E = \text{mean}(\text{dist}(\{PT_e\}_{e=1}^k, PT))$). If PT is part of an emerging class, $P(Novel|PT)$ is anticipated to be larger than $P(Existing|PT)$. All data points in the buffer associated with such PT are considered novel. When more than half of the data points in the buffer are marked as novel, an emerging class is acknowledged. Lastly, the prototypes for the newly identified class is extracted and integrated into the model. The rest of the data points are deemed as concept drifting and classified according to the process described in Section 3.3.2. If an insufficient number of novel points are identified in the buffer, they will be retained and re-evaluated when new outlier data points emerge. It is also necessary to periodically clear the buffer to eliminate data points that no longer contribute to detecting new classes. For this, any data point with an age greater than twice the block size is removed from the buffer. Algorithm 2 presents a step-by-step procedure for the detection of a novel class.

3.3.4. Online learning mechanism

In order to maintain accuracy and relevance, learning algorithms need to be updated consistently, as they must adapt to the ever-changing concepts present within the data stream. By doing so, this ensures that the model accurately reflects the current data landscape. The process of updating the model as new data blocks become accessible is detailed in Algorithm 3, which outlines the crucial steps involved. The key stages of the update procedure are as follows:

Step 1 Active and disagreement based learning: By employing active learning in streaming data, the labeling burden can be reduced significantly while maintaining or even improving the model's performance. Active learning is a technique that aims to reduce the labeling burden in streaming data by intelligently selecting the most informative and representative examples from a large pool of unlabeled data, querying an expert to label those examples, and using those labeled examples to improve the performance of a predictive model. This approach ensures that the available labeling resources are used efficiently, focusing on the samples that will yield the most significant impact on the learning process. These algorithms typically rely on heuristics, such as uncertainty sampling or diversity sampling, to select the most informative examples for labeling.

In this paper, we present a dual-criteria approach that takes into account both uncertainty and local density when selecting data points for manual labeling (line 3–12, Algorithm 3). For each data point (x) in a newly arrived block (S_k), we search for the two nearest prototypes (PT^c and $PT^{\hat{c}}$) of the c th and $\hat{c}$ th classes, respectively, at every granularity level, beginning from the top level ($\ell = 1$) and continuing until the final level is reached ($\ell = \mathcal{L}$). We then use Eq. (11) to determine whether the data point is located inside the influence area of prototypes from a single class, indicating that it is spatially close to the prototypes of the c th class but distant from those of other classes. When this is the case, we can confidently assume that x belongs to the c th class and assign it a pseudo-label of $(x, \hat{y} = c)$.

$$\begin{aligned} &\text{IF } (\|PT^c - x\|^2 \leq \Gamma_{\ell}^c) \text{ and } (\|PT^{\hat{c}} - x\|^2 > \Gamma_{\ell}^{\hat{c}}) \\ &\text{THEN (Assign predicted label } (\hat{y}), \text{ i.e., } (x, \hat{y})) \end{aligned} \quad (11)$$

However, if the condition is not met, the data point is deemed uncertain and temporarily stored in a buffer for further processing (line 13–24, Algorithm 3). Subsequently, the final level prototypes of all classes, i.e., $Model.\{PT_{\ell}^c\}_{c=1}^C$, which are representing the finer details of class distribution are utilized to generate Voronoi tessellations in the data space. This attracts nearby data points in the buffer, forming groups of similar data points (clusters). For each group (where each group is formed by PT^c), the most distant data point (the one which is more close to second nearest class prototype $PT^{\hat{c}}$) from the prototype is chosen to represent the entire group, which then requires manual labeling. After obtaining the actual label y , all data points in that group are assigned the label y . By combining these two criteria, the algorithm is able to query representative data samples with which it is uncertain.

After querying the labels, the current model evaluates each labeled data point to handle concept drift and maintain up-to-date information. The reliability of each PT^c involved in prediction are updated using a supervised criterion based on the agreement between real and predicted labels. Specifically, if an PT^c produces a correct prediction, its reliability increase and its timestamp is updated, while those of a wrongly predicted PT^c decrease. Low-accuracy PT^c s are removed from the model to enhance performance.

Step 2 Modeling gradual concept drift: Taking into account the impact of time is essential when processing data streams. As time goes on, the model may contain obsolete concepts that need to be eliminated. To ensure the model stays current with the latest trends, older prototypes are gradually phased out using exponential decay (line 25). The reliability of each prototype in the model is determined by an exponential decay formula, as shown below:

$$\{PT_{\ell}^c\}_{c=1}^C.\Omega = \{PT_{\ell}^c\}_{c=1}^C.\Omega \times 2^{-\delta \times (cY - Y)} \quad (12)$$

In this equation, the reliability, represented by Ω , initially starts at 1. As new data blocks are introduced, the weight decreases over time, based on the difference between the current time (cY) and the last update time (Y), with a decay rate of δ . In simpler terms,

Algorithm 3: Online Learning Mechanism — Part 1

```

1 Function UpdateModel( $S_k, Model, GlobalMeta, cY$ )
2    $buff \leftarrow \phi$ 
   // Selecting data points for labeling
3   foreach  $x \in S_k$  do
4     foreach  $\ell = 1, 2, \dots, \mathcal{L}$  do
5       Find  $PT^c$  and  $PT^{\hat{c}}$  two nearest prototypes of the  $c^{th}$  and  $\hat{c}^{th}$  classes, respectively, at granularity level  $\ell$ , i.e.,  $Model.\{PT_{\ell}^c\}_{c=1}^{\mathcal{C}}$ .
6       if  $(\|PT^c - x\|^2 \leq \Gamma_{\ell}^c)$  and  $(\|PT^{\hat{c}} - x\|^2 > \Gamma_{\ell}^{\hat{c}})$  then
7         Assign predicted label ( $\hat{y} = c$ ) as pseudo-label, i.e.,  $(x, \hat{y})$ 
8       else
9          $buff \leftarrow buff \cup x$ 
10      end
11    end
12  end
13   $\{\mathcal{G}_i\}_{i=1}^n \leftarrow \text{CREATEDATA CLOUDS}(buff, Model.\{PT_{\ell}^c\}_{c=1}^{\mathcal{C}})$  // Creating data groups (clusters) from buffer around last
    level prototypes
  // Querying actual labels and disagreement based learning
14  foreach  $\mathcal{G}_i \in \{\mathcal{G}_i\}_{i=1}^n$  do
15    Select a distant data point  $x \in \mathcal{G}_i$  (where  $\mathcal{G}_i$  is formed by  $PT^c \in Model.\{PT_{\ell}^c\}_{c=1}^{\mathcal{C}}$ ) // data point that is nearest to  $PT^{\hat{c}}$ 
16    Get the actual label ( $y$ )
17    Label all data points  $x_i \in \mathcal{G}_i$  as  $(x_i, y)$ 
18    if  $x.y = x.\hat{y}$  then
19       $PT^c.\Omega \leftarrow PT^c.\Omega + 1$  //  $PT^c$  consistent with true label
20       $PT^c.Y \leftarrow cY$  // increment reliability // update time
21    else
22       $PT^c.\Omega \leftarrow PT^c.\Omega - 1$  //  $PT^c$  inconsistent with true label // decrement reliability
23    end
24  end
  // Decay of reliability assigned to prototypes as time passes. see Eq. (12)
25   $\text{DECREASE RELIABILITY}(Model, cY)$ 
  // Delete obsolete prototypes
26  foreach  $\ell = 1, 2, \dots, \mathcal{L}$  do
27     $PT^{Temp} \leftarrow \text{FIND}(PT \in \{PT_{\ell}^c\}_{c=1}^{\mathcal{C}} \mid PT.\Omega \leq 0)$ 
28    foreach  $PT^c \in PT^{Temp}$  do
29      if  $\ell = \mathcal{L}$  then
30        Update class local meta parameter  $GlobalMeta.meta^c$  and upper level prototypes meta  $\{PT_{\ell}^c\}_{\ell=1}^{\mathcal{L}-1}$  // see Eq. (13)
31        and (14)
32         $\text{REMOVEPT}(PT^c)$ 
33      else
34         $\text{REMOVEPT}(PT^c)$ 
35      end
36    end
  end

```

this equation ensures that older prototypes lose their significance in the model as time passes, allowing it to stay up-to-date with recent data trends.

To remove prototypes in each class (c^{th}), we follow these steps (line 25–36 Algorithm 3): For higher level prototypes PT_{ℓ}^c starting from level $\ell = 1$ to $\mathcal{L} - 1$, we immediately delete them if their reliability (weight) approaches zero. For the last level prototypes $PT_{\mathcal{L}}^c$, before removing them, we delete their effect from the class local meta parameter $meta^c$ and upper level prototypes $\{PT_{\ell}^c\}_{\ell=1}^{\mathcal{L}-1}$. For every last level prototype $PT \in PT_{\mathcal{L}}^c$ with $\Omega(reliability) \leq 0$, we search for its nearest prototype ($\hat{PT}$) at the ℓ^{th} upper level prototypes ($\hat{PT} \in \{PT_{\ell}^c\}_{\ell=1}^{\mathcal{L}-1}$). If it lies inside the local influence area of $\hat{PT}$ (i.e., $\|PT - \hat{PT}\|^2 \leq \Gamma_{\ell}^c$), we update the local meta for $\hat{PT}$ as follows:

$$\hat{PT} = ((\hat{PT}.LS^p = \hat{PT}.LS^p - PT.LS^p), (\hat{PT}.SP^p = \hat{PT}.SP^p - PT.SP^p), (\hat{PT}.N^p = \hat{PT}.N^p - PT.N^p)) \quad (13)$$

This update will slightly relocate the prototype center. We also update the local $meta^c$ parameter containing $\Pi^c = (LS^c, SP^c, N^c)$ and $\{\Gamma_{\ell}^c\}_{\ell=1}^{\mathcal{L}}$ as follows:

$$old_mu^c = \frac{LS^c}{N^c}, \quad old_E^c = \frac{SP^c}{N^c} \quad (14a)$$

$$\Pi^c = (LS^c = LS^c - LS^p, SP^c = SP^c - SP^p, N^c = N^c - N^p) \quad (14b)$$

Algorithm 3: Online Learning Mechanism — Part 2

```

1 Function UpdateModel( $S_k, Model, GlobalMeta, cY$ )
  // This is the continuation from the previous page
  // Updating local meta parameters and existing prototypes with latest data block
2   $\{\mathcal{X}^c\}_{c=1}^C \leftarrow \text{PARTITION}(S_k)$  // Partition  $S_k$  into distinct classes
3  foreach  $\mathcal{X}^c \in \{\mathcal{X}^c\}_{c=1}^C$  do
4    Update local meta parameter  $GlobalMeta.meta^c$ , i.e.,  $\Pi^c$  and  $\{\Gamma_\ell^c\}_{\ell=1}^L$ 
5    foreach  $x \in \mathcal{X}^c$  do
6      foreach  $\ell = 1, 2, \dots, L$  do
7        Find nearest  $PT \in Model.\{PT_\ell^c\}$ 
8        if  $\|PT - x\|^2 \leq \Gamma_\ell^c$  then
9          Update  $PT$  incrementally using  $x$ 
10         else
11           Create new  $\hat{PT} \leftarrow (LS^p \leftarrow x, SP^p \leftarrow \|x\|^2, N^p \leftarrow 1, \Omega \leftarrow 1, Y \leftarrow c.Y)$ 
12            $Model.\{PT_\ell^c\} \leftarrow Model.\{PT_\ell^c\} \cup \hat{PT}$ 
13         end
14       end
15     end
16   end
17 End Function

```

$$new_mu^c = \frac{LS^c}{N^c}, \quad new_E^c = \frac{SP^c}{N^c} \quad (14c)$$

$$\{\Gamma_\ell^c\}_{\ell=1}^L = \frac{new_E^c - \|new_mu^c\|^2}{old_E^c - \|old_mu^c\|^2} \{\Gamma_\ell^c\}_{\ell=1}^L \quad (14d)$$

Step 3 Updating model with recent data: In any streaming data learning method, the latest data block is deemed to be the most informative and representative of current concepts. Therefore, the current data block (S_k) is used to update the local class meta-parameters and the structure of existing prototypes for each class (line 2–16, Algorithm 3). First, the data block (S_k) is partitioned into several classes $\{\mathcal{X}^c\}_{c=1}^C$ based on predicted and actual labels. The update process for each class $\mathcal{X}^c$ is as follows: Initially, the local meta-parameter $meta^c$ containing $\Pi^c = (LS^c, SP^c, N^c)$ and $\{\Gamma_\ell^c\}_{\ell=1}^L$ are updated according to Eq. (14). Specifically, Π^c is updated as $\Pi^c = ((LS^c = LS^c + L\hat{S}^c), (SP^c = SP^c + S\hat{P}^c), (N^c = N^c + \hat{N}^c))$, where $L\hat{S}^c = \sum_{i=1}^{\hat{N}^c} x_i$, $S\hat{P}^c = \sum_{i=1}^{\hat{N}^c} \|x_i\|^2$, and $\hat{N}^c$ is the total number of data samples in $x_i \in \mathcal{X}^c$. Moreover, for each data point $x \in \mathcal{X}^c$, if it lies within the boundary (radius) of the nearest prototype (PT) at every granularity level $PT \in \{PT_\ell^c\}_{\ell=1}^L$, the data point is added to the PT and the prototype's statistical summary (prototype meta) is further updated (i.e., $PT = ((LS^p = LS^p + x), (SP^p = SP^p + \|x\|^2), (N^p = N^p + 1), \Omega = 1, Y = c.Y))$. However, if the data point falls outside the boundary of the nearest prototype (PT), a new prototype ($\hat{PT}$) containing x is created (i.e., $\hat{PT} = (LS^p = x, SP^p = \|x\|^2, N^p = 1, \Omega = 1, Y = c.Y)$) and added to the prototype collection at a specific level of granularity ($\{PT_\ell^c = PT_\ell^c \cup \hat{PT}\}_{\ell=1}^L$).

By following these key stages, the learning algorithm will remain current and adaptive, enabling it to accurately represent the most up-to-date information available in the data stream. This iterative process is crucial for maintaining the model's accuracy and relevance over time.

In conclusion, Algorithm 4 merges all the elements and presents the pseudo-code for our approach.

Time Complexity: Algorithm 4 consists of four main components: model initialization, classification, emerging class detection, and online data maintenance. The model initialization function is performed only once, using an initial set of data points to build the model. It includes partitioning of data points into classes and extracting prototypes, with a worst-case cubic time complexity of $\mathcal{O}(N^3)$. Classification follows the nearest neighbor principle, with linear time complexity of $\mathcal{O}(N^{PT})$. Emerging class detection involves various steps leading to a worst-case quadratic time complexity, that is, $\mathcal{O}(\hat{N}^2)$. The model update component involves data selection, creating data clouds, querying labels, decaying reliability, and updating parameters. In the worst case, its complexity simplifies to $\mathcal{O}(C \times K \times L)$, making the algorithm's time complexity dependent on the number of classes, block size, and the maximum levels of granularity.

Space Complexity: A prototype (PT) is represented by a 5-tuple and includes components such as LS^p , SP^p , N^p , Ω , and T . These components store information related to the linear sum, scalar products, number of points, weight, and time of the prototype. The space complexity for a single prototype is determined by the dimension of the linear sum vector and is denoted as $\mathcal{O}(LS_{1 \times d} + 4)$. The total space complexity considers the number of prototypes for each class (N^{PT}) and the number of classes (C), resulting in a complexity of $\mathcal{O}(C \times N^{PT} \times \mathcal{O}(PT))$. Additionally, meta information for each class is maintained, comprising a 3-tuple $\Pi^c = (LS^c, SP^c, N^c)$ and a vector $\{\Gamma_\ell^c\}_{\ell=1}^L$ of dimension L . The space complexity for the global meta information ($GlobalMeta$) considering all classes is $\mathcal{O}(C \times (LS_{1 \times d} + \Gamma_{1 \times L} + 2))$.

Algorithm 4: Data driven prototype framework for classification and novel class discovery

Input: S : Streaming data i.e., $S = \{S_1, S_2, \dots, S_k, \dots\}$; S_{init} : Initial training data; $\mathcal{K}$: Block size; β : Buffer size; $\mathcal{L}$: Max. levels; δ : Decay rate

Output: Classification Results Predicted Labels: $\hat{y}$

```

1  $B \leftarrow \phi$ 
2  $[Model, GlobalMeta] \leftarrow \text{MODELINIT}(S_{init}, \mathcal{L})$  // see Algorithm 1
3 while  $S \neq \phi$  do
4   Read ( $S_k$ ) // recent data block
5   foreach data point  $x \in S_k$  do
6     // see Definition 1: Outlier
7     if  $\text{Outlier}(x) = \text{true}$  then
8        $B \leftarrow B \cup x$ 
9     else
10       $\hat{y} \leftarrow \text{CLASSIFY}(x)$  // see Section 3.3.2
11      Display output  $\hat{y}$ 
12      Save output:  $plabel.add(\hat{y})$ 
13    end
14  end
15  if  $|B| \geq \beta$  then
16     $[NvList, ExList] \leftarrow \text{CLASSDISC}(B, Model, GlobalMeta, \mathcal{L})$  // See Algorithm 2
17    Display  $eNvListe$ 
18    foreach  $x \in ExList$  do
19       $\hat{y} \leftarrow \text{CLASSIFY}(x)$  // see Section 3.3.2
20      Display output  $\hat{y}$ 
21      Save output:  $plabel.add(\hat{y})$ 
22    end
23  end
24   $cY \leftarrow currTime$ 
25   $S_k \leftarrow (S_k - B) \cup ExList$ 
26   $\text{UPDATEMODEL}(S_k, Model, GlobalMeta, cY)$  // See Algorithm 3
27 end

```

4. Experiment design

This section provides a comprehensive overview of the experimental setup, including the datasets used for evaluation, the competing methods for streaming data classification, and an insightful analysis of our experimental results. Our primary aim is to robustly validate the efficacy of our proposed algorithm.

4.1. Datasets

To validate the performance and efficiency of our proposed algorithm, we have selected a combination of eleven datasets. Three of these are synthetic datasets, generated under various settings, and the remainder are derived from real-world scenarios. The synthetic datasets, utilized in several research studies such as [Haque et al. \(2016\)](#), have been specifically designed to simulate the presence of concept drift. All features of the chosen datasets have been normalized to fall within the range of (0, 1). A detailed description of each dataset, outlining their key characteristics, can be found in [Table 2](#).

Real datasets: The following real datasets used in our experimental setup: The *Shuttle (STL)*¹, *Gas Sensor Array Drift (GSD)*¹, *Spam*², *IoT Botnet Attack (IoT)*¹, *KDDCup (KDD)*¹, *Sensor Less Drive Diagnosis (SLDD)*¹, *Human Activity Recognition (HAR)*¹, *Pen-Based Recognition of Handwritten Digits (PIN)*¹, *Occupancy*¹, *Zoo*³, *BNG_Lymph*³, *BNG_Wine*³, *Electricity*¹.

Synthetic datasets: In our study, we employ various synthetic datasets, including the Rotating Hyperplane dataset ([Narasimhamurthy & Kuncheva, 2007](#)). We also generate multiple variations of the RandomRBFGeneratorDrift (RBF) dataset using the MOA (Massive Online Analysis) framework, denoted as RBF1, RBF2, RBF3, RBF4, and RBF5. RBF1, RBF2, and RBF3 exhibit a drift speed of 0.001, while RBF4 and RBF5 have a faster drift speed of 0.003. Additionally, we utilize the MOA dataset generated using the RandomRBFGeneratorDrift, which simulates concept drift and evolution and has been used in previous studies like [de](#)

¹ <https://archive.ics.uci.edu/ml/datasets.php>

² http://mlkd.csd.auth.gr/concept_drift.html

³ <http://www.people.vcu.edu/~acano/KUE/datasets.zip>

Table 2
Overview of the characteristics of datasets utilized for performance evaluation.

#	Data set	#Features	#Class	Size	Max class %	Min class %	Drift type
1	Shuttle	9	7	58,000	78.60	0.02	Unknown
2	GSD	128	6	13,910	21.63	11.80	Unknown
3	Spam	500	2	9,324	74.40	25.60	Unknown
4	IoT	115	9	663,795	18.47	4.38	Unknown
5	KDDcup	34	23	494,021	56.84	0.00	Unknown
6	SLDD	48	11	58,509	9.09	9.09	Unknown
7	HAR	561	6	10,299	18.88	13.65	Unknown
8	Pin Digits	16	10	10,992	10.40	9.59	Unknown
9	Occupancy	5	2	20,560	76.89	23.10	Unknown
10	Zoo	17	7	1,000,000	39.62	4.29	Unknown
11	BNG_lymph	19	4	1,000,000	54.34	1.65	Unknown
12	BNG_Wine	14	3	1,000,000	40.10	27.75	Unknown
13	Electricity	8	2	45,312	57.54	42.45	Unknown
14	Hyp	2	2	100,000	50.03	49.97	Gradual
16	RBF1	100	5	200,000	21.25	18.67	Gradual
17	RBF2	1000	5	200,000	20.88	19.21	Gradual
18	RBF3	2000	5	200,000	20.50	19.46	Gradual
19	RBF4	100	5	200,000	21.36	18.78	Gradual
20	RBF5	1000	5	200,000	20.98	19.04	Gradual
21	Stagger	3	2	200,000	66.39	33.40	Gradual
22	Waveform1	21	3	400,000	33.44	33.12	Gradual
23	Waveform2	21	3	400,000	33.54	33.17	Sudden
24	SEA1	3	2	400,000	61.33	38.66	Gradual
25	SEA2	3	2	400,000	59.60	40.40	Sudden
26	MOA	4	4	100,000	25.40	23.51	Gradual
27	SENNE	2	3	20,000	33.00	33.00	No drift

Faria, Ponce de Leon Ferreira Carvalho, and Gama (2016). Furthermore, Stagger, SEA, and Waveform datasets are also generated using MOA. Lastly, SENNE is employed as described in Cai et al. (2019).

4.2. Comparison algorithms

Semi-Supervised methods: The following methods are used: ReSSL (Shao, Huang, Yang, & Luo, 2016), SPASC (Hosseini et al., 2016), ORSL (Din et al., 2020), DSL (Din et al., 2022), CAL (Nguyen, Gomes, & Bifet, 2019), and COCEL(COC),(Fahy, Yang, & Gongora, 2023) which are cluster-based models. TLP (Wagner et al., 2018) is a graph-based method. Similarly, ST (Nguyen et al., 2019) is a self-training model and has been integrated into the MOA software (Bifet, Holmes, Kirkby, & Pfahringer, 2010). Lastly, DEV DAN (DEV) (Ashfahani, Pratama, Lughofer, & Ong, 2020) is based on deep evolving denoising autoencoder.

Supervised methods: In order to compare supervised data stream classification methods, we have chosen four state-of-the-art algorithms from recent literature. *Accuracy Updated Ensemble (AUE)* (Brzezinski & Stefanowski, 2014), *SAMkNN* (Losing, Hammer, & Wersing, 2018), *Adaptive Random Forest (ARF)* (Gomes et al., 2017), *Streaming Random Patches (SRP)* (Gomes, Read, & Bifet, 2019), and *ROSE* (Cano & Krawczyk, 2022).

Algorithm Settings: The parameter settings for our proposed algorithm are as follows: (i) initial training set size, $S_{init} = 2000$, (ii) block size, $\mathcal{K} = 1000$ for synthetic datasets and $\mathcal{K} = 2000$ for real datasets, (iii) buffer size, $\beta = 250$, (iv) maximum levels, $\mathcal{L} = 5$, and (v) decay rate, $\delta = 0.0005$. The source code for our proposed algorithm can be accessed via the provided link.⁴ The authors have made the source codes for ReSSL, SPASC, ORSL, and DSL publicly available. For the implementation of TLP and COC, we adhered strictly to the details provided in the corresponding paper. The parameter settings for these algorithms were configured as follows: In SPASC, the chunk size was tuned within the range of {50, 100, 500, 1000}, and the number of clusters was selected from the range {5 – 50}, incrementing in steps of 5. For TLP, the window size was configured to hold 1000 data points. The parameters for ReSSL, ORSL, DSL, CAL, and ST are set in alignment with the recommendations specified in the original paper. All supervised algorithms mentioned here are included in the Massive Online Analysis (MOA) framework (Bifet et al., 2010). The parameters for these algorithms have been set to the default values as outlined in the MOA framework.

4.3. Analysis of results

This section provides a detailed analysis of the conducted experiments across all chosen datasets. It delves into a thorough exploration of our proposed algorithm, comparing its performance with several leading techniques available in the field. Lastly, we conduct a sensitivity analysis, scrutinizing the impact of various parameters on the effectiveness of our proposed method.

⁴ <https://shorturl.at/btDIO>

Table 3

Comparative analysis of classification performance (average accuracy) for the proposed method and various semi-supervised classifiers on various datasets.

Datasets	Algorithms										Query
	Our	ReSSL	SPASC	ORSL	TLP	DSL	CAL	ST	DEV	COC	Label%
Shuttle	99.03	96.63	95.21	98.01	94.22	97.26	93.61	94.74	88.52	97.51	2%
GSD	95.36	89.64	57.22	93.87	74.79	93.14	58.88	58.84	56.25	86.60	15%
Spam	92.75	86.63	78.48	93.54	70.47	94.61	86.20	86.54	91.80	88.23	17%
IoT	99.52	93.52	90.46	98.63	93.56	96.54	76.21	77.14	98.56	95.72	7%
KDDcup	98.24	95.30	96.45	94.05	78.14	95.40	98.75	95.50	98.92	95.28	1%
SLDD	94.49	82.87	42.51	93.76	54.79	80.29	74.58	77.94	42.89	80.66	24%
HAR	90.23	85.52	69.80	88.55	62.84	87.34	36.39	37.13	32.69	83.17	12%
Pin Digits	98.35	92.68	83.54	94.58	76.91	90.74	85.91	82.75	64.51	92.38	15%
Occupancy	98.48	97.06	95.24	98.16	96.15	96.78	92.55	93.37	94.76	96.77	13%
Zoo	92.59	88.98	85.31	90.74	88.27	87.76	85.91	84.73	93.97	87.44	12%
BNG_lymph	80.71	77.49	72.58	78.11	76.34	76.80	72.52	73.64	90.00	76.28	17%
BNG_Wine	90.03	88.30	81.66	88.76	85.47	86.34	84.99	86.04	91.95	86.73	15%
Electricity	66.52	64.52	58.61	72.68	57.54	54.89	51.34	52.07	62.16	58.91	20%
Hyp	97.14	96.36	57.07	96.36	95.78	95.69	85.65	88.87	95.52	92.83	7%
RBF1	99.88	90.65	70.88	91.59	76.25	91.59	88.57	90.68	28.04	89.07	11%
RBF4	92.03	85.66	67.48	84.28	67.58	83.82	80.37	81.98	24.75	82.59	10%
Stagger	100.00	99.16	96.76	100.00	99.57	97.84	96.66	97.37	100.00	99.57	1%
Waveform1	80.79	76.55	70.34	77.94	74.25	74.69	75.40	75.37	77.70	76.37	16%
Waveform2	80.12	77.33	72.37	76.88	73.61	73.80	74.64	75.89	71.54	73.28	18%
SEA1	81.05	79.96	76.18	85.95	80.39	76.81	75.24	74.77	86.20	77.48	20%
SEA2	80.50	77.62	77.76	84.33	79.07	74.88	74.47	75.08	87.05	75.67	20%
Avg Acc	90.85	86.78	76.00	89.56	78.86	86.05	78.52	79.07	75.13	85.36	
Avg Rank	1.50	4.13	8.00	2.63	7.25	4.44	8.00	7.44	5.43	5.52	

Table 4

Comparative average F1-Scores attained by competing semi-supervised methods on various datasets.

Datasets	Algorithms									
	Our	ReSSL	SPASC	ORSL	TLP	DSL	CAL	ST	DEV	COC
Shuttle	42.72	38.11	37.28	40.59	35.44	39.82	35.54	36.70	37.94	36.29
GSD	94.42	87.84	58.12	93.08	71.62	92.58	56.79	57.23	51.17	85.33
Spam	88.06	67.78	73.57	87.27	66.75	89.26	81.13	81.50	89.18	82.83
IoT	99.46	90.72	91.85	98.60	91.67	98.70	77.25	78.40	98.91	94.54
KDDcup	24.34	18.27	23.49	20.45	8.27	22.64	36.87	31.87	31.37	23.11
SLDD	94.12	80.21	41.87	92.65	50.19	78.37	73.82	75.39	41.48	79.52
HAR	90.34	88.38	72.84	88.07	51.73	86.19	35.91	36.45	32.06	80.07
Pin Digits	98.04	91.90	82.97	92.34	75.33	88.96	85.09	81.28	61.59	91.24
Occupancy	98.48	97.20	96.54	98.04	95.80	95.76	90.57	92.54	92.86	95.66
Zoo	85.70	81.33	78.54	84.75	81.54	79.48	78.64	77.76	88.22	80.73
BNG_lymph	69.44	66.22	60.11	66.73	65.93	65.88	63.67	64.28	83.41	65.29
BNG_Wine	90.08	87.59	80.95	87.07	85.11	85.60	83.34	85.85	91.89	85.94
Electricity	61.24	55.87	50.29	66.76	51.49	50.15	50.64	51.07	60.40	53.37
Hyp	97.08	95.90	50.67	96.20	94.86	95.07	83.55	87.30	95.50	91.26
RBF0.001	99.87	88.05	67.62	90.77	73.31	90.97	86.73	89.68	21.63	87.42
RBF0.003	87.77	84.60	66.59	83.90	65.18	83.83	78.46	80.44	17.68	80.39
Stagger	100.00	99.05	96.57	100.00	99.41	96.83	96.12	96.74	100.00	99.06
Waveform1	80.23	75.73	70.19	76.82	74.05	73.89	74.27	75.11	77.65	74.46
Waveform2	80.00	76.38	71.94	75.64	73.26	73.43	73.89	75.03	71.06	71.82
SEA1	78.25	76.22	74.53	84.67	79.38	75.21	71.29	72.76	85.17	75.16
SEA2	77.86	76.89	77.09	83.24	78.36	73.64	73.87	74.51	86.10	74.73
Avg F1-Socre	82.74	77.34	67.79	81.32	69.94	77.92	70.83	71.52	67.39	76.58
Avg Rank	1.56	4.44	7.69	3.00	7.38	4.88	7.81	6.94	5.63	5.69

4.3.1. Performance comparison

The classification performance of our algorithm is primarily evaluated based on accuracy and the F1-score, utilizing the interleaved test-then-train evaluation approach. This method tests and subsequently updates the classifier using an incoming data block, thereby providing real-time performance assessment and continuous adjustments. Initially, all classifiers are trained on a labeled set consisting of 2,000 instances ($S_{init} = 2000$ instances).

Throughout the entire data stream, accumulated accuracy is measured, offering a comprehensive overview of the performance. To ensure the robustness and reliability of the results, each experiment is carried out ten times, with the average accuracy and F1-score reported.

Table 5

Comparative analysis of average accuracy and F1-Score for the proposed method and various supervised classifiers on various datasets.

Datasets	Accuracy						F1-Score					
	Our	ARF	SAMkNN	SRP	AUE	ROSE	Our	ARF	SAMkNN	SRP	AUE	ROSE
Shuttle	99.03	99.42	99.35	99.55	96.64	99.39	42.72	58.43	42.44	56.54	51.25	57.36
GSD	95.36	86.43	94.90	87.73	59.17	65.02	94.42	85.64	94.37	86.51	58.82	63.72
Spam	92.75	90.06	88.80	93.66	79.37	92.95	85.06	85.94	84.24	91.06	76.96	90.08
IoT	99.52	99.63	99.38	99.45	83.83	99.51	99.46	99.66	99.12	99.26	87.32	99.14
KDDcup	98.24	99.30	99.24	99.25	97.99	99.26	24.34	40.90	38.94	50.64	42.86	48.57
SLDD	94.49	93.22	95.15	89.46	80.12	81.04	94.12	93.21	96.13	89.48	80.02	80.64
HAR	90.23	54.36	91.69	60.54	32.16	42.32	90.34	53.31	91.15	57.84	45.64	44.37
Pin Digits	98.35	89.62	96.51	90.95	75.95	88.11	98.04	89.30	96.15	90.75	75.61	87.86
Occupancy	98.48	98.58	98.78	98.68	87.48	98.50	98.48	98.54	98.65	98.53	98.42	98.42
Zoo	92.59	91.67	84.19	93.37	90.98	91.96	85.70	84.51	79.38	88.37	84.74	84.73
BNG_lymph	80.71	91.34	84.34	91.20	89.40	90.36	69.44	81.94	71.97	81.49	79.55	80.37
BNG_Wine	90.03	94.30	72.84	94.35	93.00	92.79	90.08	93.91	71.77	93.93	92.67	91.80
Electricity	66.52	78.84	65.84	79.85	76.71	76.82	61.24	76.34	66.63	78.61	74.37	75.61
Hyp	97.14	90.09	98.58	98.42	96.90	97.28	97.08	89.87	98.11	98.30	96.77	96.62
RBF1	99.61	94.30	99.18	99.78	84.74	95.64	99.24	94.77	99.20	99.61	83.65	94.16
RBF4	92.03	92.20	98.53	91.06	65.27	90.76	87.77	91.90	98.40	90.89	64.75	90.24
Stagger	100.00	99.99	99.98	100.00	98.33	99.99	100.00	99.99	99.99	100.00	98.54	99.99
Waveform1	80.79	82.28	83.67	83.58	82.07	83.74	80.23	83.24	83.15	83.34	83.05	83.56
Waveform2	80.12	83.61	83.93	83.21	82.61	84.23	80.00	83.34	83.26	83.21	82.33	83.90
SEA1	81.05	87.84	87.55	85.80	87.52	84.41	78.25	86.22	86.54	84.53	86.73	83.67
SEA2	80.50	88.59	86.68	86.54	88.03	89.22	77.86	87.87	85.39	85.16	87.75	88.67
Avg Acc	90.84	89.79	90.91	90.78	82.30	87.78	82.57	83.75	84.05	85.15	77.70	82.07
Avg Rank	2.73	3.27	2.64	2.64	5.91	3.82	3.00	3.27	3.00	2.36	5.27	4.09

Comparison with semi-supervised methods: In the case of semi-supervised learning algorithms, following the initial training phase, the remaining data stream is partially labeled at random. After a data block is classified, the true labels (when available) are revealed to the learning algorithms, which then use this information to update their classifiers.

First we present a comprehensive comparison of our proposed algorithm against seven other SSL algorithms for data streams. [Tables 3 and 4](#) respectively provide the average classification accuracy and the F1-score for all datasets under varying percentages of labeled data. The superior results are highlighted in bold. A careful analysis of [Table 3](#) reveals that our algorithm consistently outperforms the competition, achieving an impressive average classification accuracy of 95% across all datasets. This score places our algorithm at the top among all evaluated methods. Notably, the classification performance of our algorithm exceeds that of the second-best algorithm by an average of 3 – 5%. Turning to [Table 4](#), it becomes evident that our algorithm consistently achieves high F1-scores, again surpassing other algorithms on most datasets. These results underscore the efficiency and effectiveness of our proposed method in comparison to other available algorithms. Our algorithm's notable strength lies in its adaptability and its disagreement-based learning approach, making effective use of both labeled and unlabeled data to improve predictive performance, particularly in environments with concept drift in streaming data. Critically, our algorithm shines when the labeled data is sparse, consistently outperforming other algorithms in these conditions.

Comparison with supervised methods: For supervised learning algorithms, once the class labels of the incoming data block have been predicted, the true class labels are made immediately available. This real-time feedback allows the classifiers to be updated promptly, maintaining their accuracy.

[Table 5](#) presents the overall average performance of our proposed algorithm in terms of accuracy and F1-score, as tested on various datasets. It is evident from the table that even with different percentages of labeled data, our algorithm consistently exhibits competitive predictive performance in comparison to supervised algorithms that utilize 100% labeled data. For instance, in the KDDcup, Shuttle, and IoT datasets, our algorithm matches the performance of supervised algorithms with only 1%, 2%, and 7% of labeled data, respectively. Further analysis indicates that our algorithm either matches or surpasses the performance of other algorithms even when the labeled data is within the 5 – 15% range. This is testament to the superior classification performance of our method, particularly when working with limited labeled data, compared to a range of other semi-supervised and supervised algorithms. This performance indicates a decreased reliance on labeled data, marking our algorithm as an efficient and effective tool for mining data streams that exhibit concept drift. [Fig. 4](#) presents a comparison of the classification accuracy achieved by various supervised algorithms and the proposed methods over the entirety of the data streams for the GSD, SLDD, RBF4, and HAR datasets.

In order to ascertain the statistical significance of the performance differences between our proposed algorithm and its competitors, we employ the Friedman rank-sum test. In this context, the null hypothesis is defined as “no significant difference exists among the algorithms under comparison”. If the results indicate a rejection of the null hypothesis, we proceed with the Nemenyi post-hoc test to pinpoint the exact points of difference. As a nonparametric statistical test, the Friedman rank-sum test has been performed on the data shown in [Tables 3 and 5](#), at a confidence level of 95%. This yielded p-values of $1.43e^{-17}$ and $1.12e^{-4}$ respectively for all datasets, well below the standard threshold of 0.05. This indicates that the performance of our proposed algorithm significantly differs from the competing algorithms from a statistical standpoint. Upon the rejection of the null hypothesis, the Nemenyi post-hoc test is employed to construct a critical difference diagram (as shown in [Fig. 5](#)), allowing us to visualize and understand the specific differences between the performances of each algorithm.

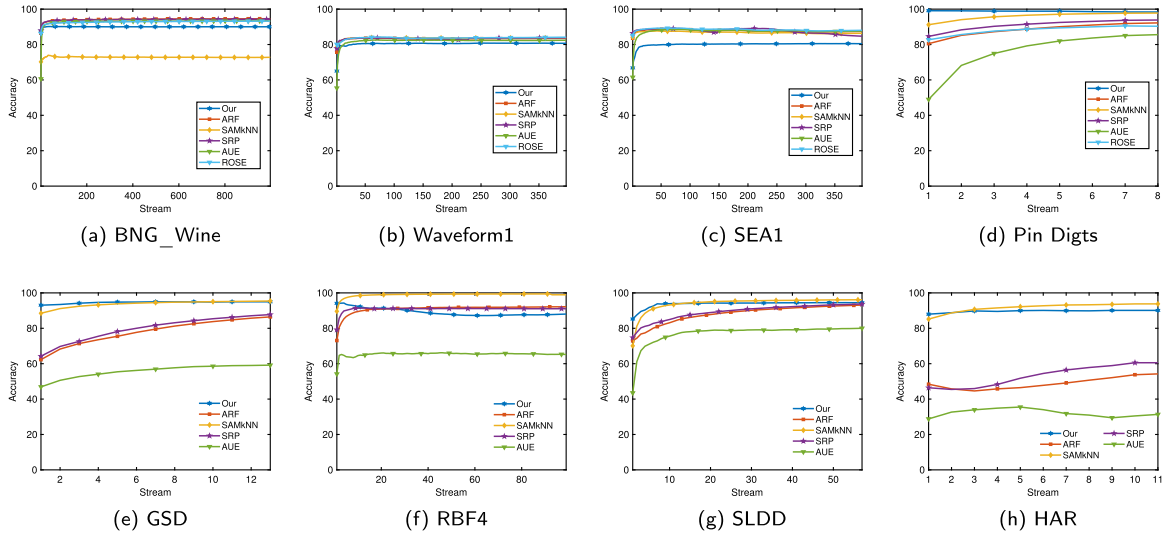


Fig. 4. Performance of different supervised algorithms and our proposed methods, displaying their classification accuracy across the entire data streams.

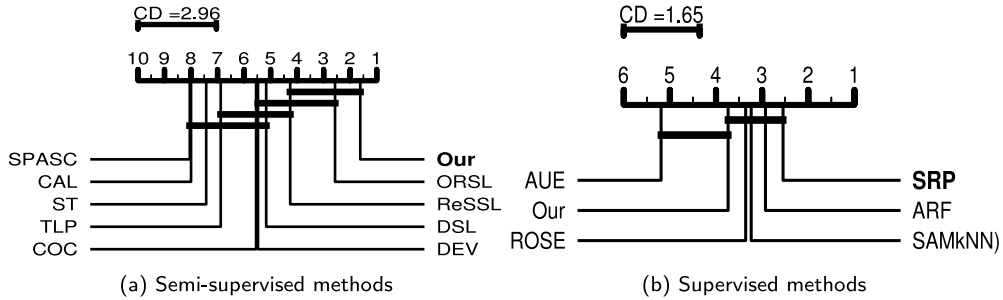


Fig. 5. Critical difference diagram depicting the Nemenyi test results at a 95% confidence level across various (a) semi-supervised (Table 3) and (b) supervised (Table 5) methods tested on all datasets.

4.3.2. Handling high dimensional data

Often referred to as the curse of dimensionality, poses a significant challenge that can substantially hinder the effectiveness of learning algorithms. As the attribute count in a dataset increases, the algorithms' accuracy and performance tend to degrade. Furthermore, heightened dimensionality escalates both space requirements and processing time for these learning algorithms. To tackle this issue and improve data representation within a compact feature space, we deploy a denoising autoencoder (DAE). This unsupervised learning technique assists us in extracting valuable and robust features that can help to mitigate the issues associated with high dimensionality. A DAE learns to generate these robust features by reconstructing input data that has been intentionally partially corrupted. The DAE operates by encoding the corrupted input into a hidden representation, and then decoding this representation back into a reconstruction of the original, uncorrupted input.

We train our DAE using an initial set of unlabeled data. Once trained, the weights of the DAE remain fixed throughout the process of streaming data. This is to avoid the time-consuming process of updating DAE weights during streaming data processing, which would necessitate backpropagation every time new data becomes available. It is important to note that each incoming data block is first transformed into the latent space before being fed into the algorithm. Parameters for the DAE are set as follows: (i) Additive Gaussian noise (σ^2) is set to 1.1. This noise is used to partially corrupt the input data as part of the denoising autoencoder process. (ii) The number of hidden layers and their sizes depend on the number of input features (d) in the dataset. For datasets with $d \geq 500$, we use 3 hidden layers. The sizes of these layers decrease progressively: $h_1 = \frac{2}{3}d$, $h_2 = \frac{1}{3}h_1$, and $z = h_3 = \frac{1}{3}h_2$. For datasets with $d < 500$, we use 2 hidden layers, with sizes $h_1 = \frac{2}{3}d$ and $z = h_2 = \frac{1}{3}h_1$. Here, h_i represents the number of neurons in the i th hidden layer, and z (which is much less than d) denotes the final reduced feature set after applying the denoising autoencoder.

Fig. 6 illustrates the impact of feature reduction on three key parameters: processing time, accuracy, and the number of prototypes. One prominent observation, as shown in Fig. 6(a), is the substantial decrease in processing time, especially for datasets with high dimensionality. By reducing the dimensionality, we can alleviate the computational cost, expedite processing times noticeably, and require less storage space. This efficiency gain is particularly advantageous when working with large-scale datasets or when computational resources are limited.

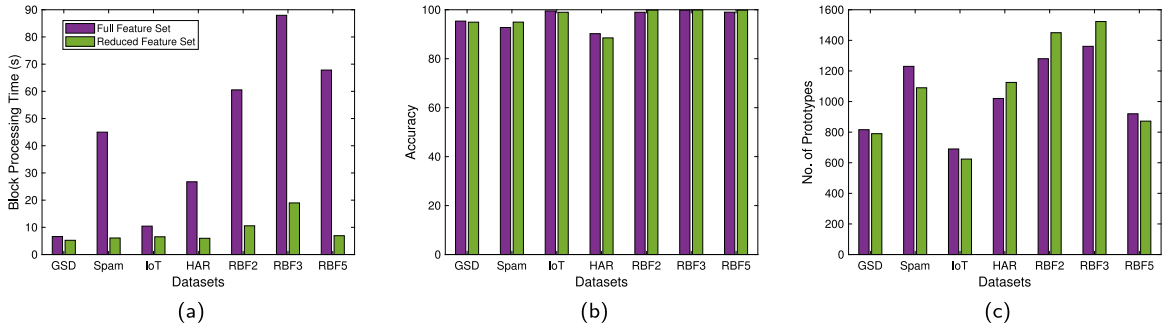


Fig. 6. The influence of feature reduction on (a) processing time, (b) accuracy, and (c) the number of prototypes.

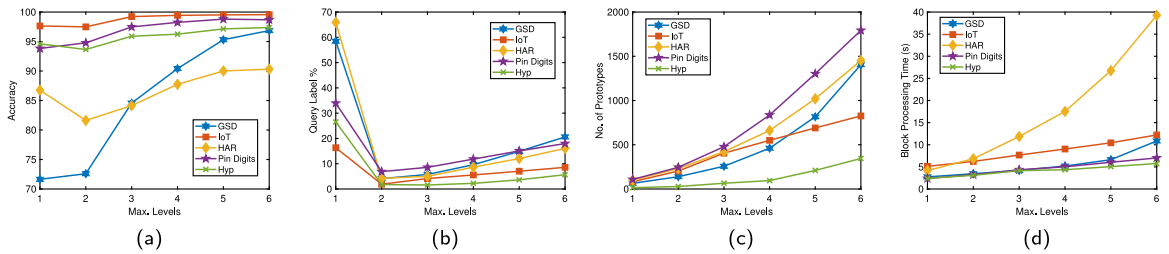


Fig. 7. Analysis of the impact of the maximum levels parameter (L) on model performance and complexity. Effects of varying L on (a) prediction accuracy, (b) label query frequency, (c) the number of maintained prototypes, and (d) model complexity. .

Figs. 6(b) and 6(c) suggest that accuracy and the number of prototypes remain largely unaffected by feature reduction. This observation emphasizes the effectiveness of the dimensionality reduction technique employed; it successfully maintains essential information and patterns despite the reduction in data attributes, thereby preserving the quality of predictions.

4.3.3. Influence of parameters

First, we provide an analysis of the effects of the parameter L (see Fig. 7), representing the maximum levels, on various aspects of our model. These aspects include prediction accuracy (Fig. 7(a)), frequency of label querying (Fig. 7(b)), the quantity of maintained prototypes (Fig. 7(c)), and the overall complexity of the model (Fig. 7(d)). To observe these effects, we conducted a series of tests on all datasets, varying the value of L within a range of 1 to 6.

Inherent in the hierarchical structure of our model, the influence of L varies across different levels. At the highest level of the hierarchy, where L is minimal, the model typically maintains a limited number of prototypes. These prototypes are highly descriptive and encapsulate the global patterns in the data, thereby providing a general overview of the data structure and a coarser classification. As we descend the hierarchy, increasing the value of L , the number of prototypes proliferates, enhancing the model's classification performance. These additional prototypes align more closely with individual raw data samples, providing a more detailed and granular representation of the data. Therefore, adjusting L allows us to control the level of abstraction or granularity in our data representation, which has potential implications for the model's performance and complexity.

Further into our analysis, we discuss the consequences of setting a large value for L . When L is excessively large, the model tends to yield an extremely fine-grained data partitioning. In extreme cases, every individual data sample could be recognized as a prototype at the lower levels of the hierarchy. Increasing L , and consequently the number of levels in the model, results in a more time-consuming learning process. This is because the model needs to generate, maintain, and update a larger number of prototypes, each representing a smaller segment of the data. It is, however, crucial to note the existence of a limit to the impact of increasing L . Once the model reaches a point where each data sample is already represented by a prototype, adding extra levels will not lead to a more detailed data partitioning. Thus, further increases in L at this stage will not significantly affect the training time or the classification performance of the model.

Figs. 8 and 9 illustrate the impact of the block size parameter (K) and decay rate (δ), respectively. We carried out a series of experiments using varied values for each parameter on several datasets: shuttle, IoT, SLDD, Hyp, and RBF4. Upon examination of these figures, it becomes evident that the variation in block size and decay rate exerts a minimal influence on the real datasets (shuttle, IoT, and SLDD). However, a markedly different effect can be observed on synthetic datasets (Hyp, RBF4). These synthetic datasets are characterized by concept drift, introduced at regular intervals. In this context, it appears that a smaller block size

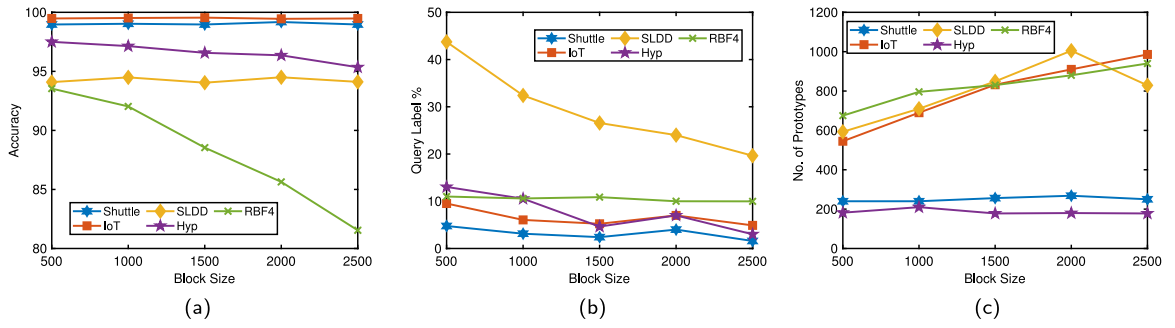


Fig. 8. The influence of block size (K) on (a) accuracy, (b) query label %, and (c) the number of prototypes.

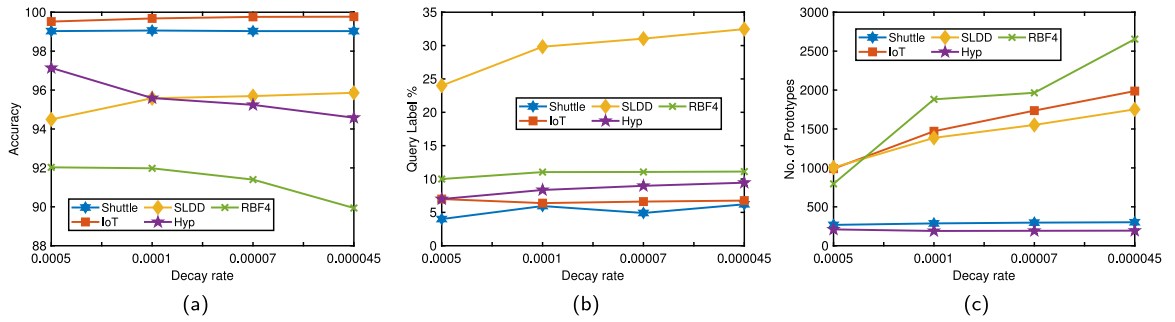


Fig. 9. The influence of decay rate (δ) on (a) accuracy, (b) query label %, and (c) the number of prototypes.

consistently yields superior performance, while an increase in block size gradually diminishes the model's performance. This trend can be attributed to the model's outdated nature after prolonged periods. Consequently, the model is unable to effectively capture and incorporate the evolving concepts introduced in the synthetic data streams. Another important observation from these experiments is that the frequency of querying labels decreases as the block size increases. This indicates that larger block sizes could potentially reduce the need for frequent label updates, thereby improving the efficiency of the learning algorithm in handling larger datasets or faster data streams.

4.3.4. Novel class detection performance

This section delves into the examination of the proposed method designed to identify novel or emerging classes within the data stream.

Datasets: We evaluate the effectiveness of our proposed method on the SENNE, MOA, and specific subsets of the KDD, Shuttle, IoT datasets, as outlined in Table 2. The aim of this evaluation is to gauge how well the method predicts and detects emerging classes. We base this assessment on the presumption that an initial training set, comprised of known classes, is available for initializing the model. This simulates the manifestation of emerging classes within a data stream. As the data stream progresses, data points associated with both known and novel classes uniformly surface throughout the stream. The expectation is that the procedure in place will update its model to incorporate both known and novel classes during this initial phase. In the subsequent phase, data points corresponding to the three classes observed in the initial phase are introduced uniformly.

Evaluation Metrics: This paper adopts two metrics to appraise the effectiveness of the proposed method. The first measure is Accuracy, computed using the formula $Acc = \frac{Acc_{knn} + Acc_{nov}}{N}$, where Acc_{knn} and Acc_{nov} denote the number of data points correctly recognized as part of the known and novel classes, respectively, and N signifies the total number of data points. The second measure is the Macro-averaged $F1$ performance, which amalgamates the recall (R) and precision (P) metrics reflecting the detection performance for each class. It is formulated as $F1 = \frac{2 \times P \times R}{(P + R)}$.

Comparison with existing methods: The evaluation process pits our proposed technique against four existing state-of-the-art methods developed to detect emerging classes. ECSMiner (Masud, Gao, Khan, Han, & Thuraisingham, 2010), the first method, is a k-NN type cluster-based ensemble classifier that incorporates emerging class detection. Similarly, SAND (Haque et al., 2016) is a semi-supervised version of ECSMiner. The third method, SENCForest (Mu, Ting, & Zhou, 2017), comprises an ensemble of decision trees generated randomly from the input data. MINAS (de Faria et al., 2016) and EMC (Din & Shao, 2020), the fourth and fifth methods, are single clustering-based techniques, performing dual functions of classification and identification of emerging classes. The performance of the proposed technique, depicted in Table 6, demonstrates that it outperforms the competing methods, consistently maintaining high accuracy levels in the classification and detection of emerging classes.

Table 6
Performance of novel class detection on various data subsets. Datasets(known classes)(new classes).

Datasets	Metric	Our	ECS	SENCf	MINAS	EMC	SAND
SENCE(1,3)(2)	Acc	99.93	98.28	98.50	98.34	98.54	99.34
	F1	99.50	98.05	98.46	98.33	98.51	99.06
MOA(0,1)(2,3)	Acc	99.79	95.87	94.28	96.48	97.47	99.67
	F1	99.05	90.58	88.67	91.87	94.21	99.29
IoT(1,3,4)(8,9)	Acc	95.90	92.81	90.68	88.75	93.67	91.57
	F1	88.56	85.24	85.44	82.19	89.33	82.61
IoT(2,4,7,8)(1,6)	Acc	99.60	93.69	89.29	86.79	95.98	98.52
	F1	99.34	91.45	84.16	81.67	93.75	97.87
IoT(1,3,7,8,9)(4,2,5)	Acc	97.66	90.47	89.67	86.47	93.73	90.33
	F1	87.59	82.57	84.57	78.33	85.41	83.75
Shuttle(1,4)(5)	Acc	99.33	96.54	94.92	97.67	98.69	98.45
	F1	99.23	95.74	92.66	95.37	97.48	97.68
KDD(1,2,3)(4)	Acc	90.93	79.88	80.07	76.47	78.74	88.67
	F1	78.54	66.70	68.15	61.75	66.09	76.16
KDD(1,2,3)(7)	Acc	96.68	90.51	89.34	86.07	89.57	93.83
	F1	88.36	86.22	83.97	80.28	84.66	86.49
KDD(1,2,3)(8)	Acc	98.04	88.91	89.67	86.47	88.04	97.08
	F1	94.87	83.50	85.31	80.30	84.91	92.81
KDD(1,2,3)(10)	Acc	98.82	90.55	87.14	86.77	91.22	96.69
	F1	93.98	85.33	80.86	82.94	87.43	91.64

5. Conclusion

In this study, we explored into the challenges of concept drift and evolution, and label availability and scarcity in data streams. In response to these challenges, we introduced a data-driven prototype-based semi-supervised learning framework. This approach innovatively captures hidden local cluster structures, offering superior data abstraction by maintaining a dynamic set of prototypes. These prototypes are data driven, and can be updated incrementally in a single pass, provide an intuitive representation of incoming streaming data. Moreover, our learning model dynamically assesses the reliability of these prototypes, enabling rapid adaptation to local concept drift and effective identification of emerging classes. By incorporating active learning, we substantially reduced the requirement for manual labeling without compromising the model's performance. In fact, we often saw performance enhancements. This strategy for label selection is a dual-criteria approach, taking into account both uncertainty and local density. In order to assess the efficacy of our approach, we conduct empirical evaluations on several benchmark datasets. We compare the results in terms of classification performance and the detection of emerging classes with those achieved by current state-of-the-art methodologies. This comparison provides a clear picture of the improvements our approach brings to the field of data stream learning. Looking forward, our research will concentrate on the development of a drift detection mechanism that utilizes both labeled and unlabeled data. This approach will ensure a prompt response to concept drift, particularly in scenarios characterized by high drift rates. Simultaneously, we plan to explore effective methodologies for incrementally updating the weights of the Denoising Autoencoder (DAE) as the data stream evolves.

CRedit authorship contribution statement

Salah Ud Din: Writing – original draft, Conceptualization, Methodology, Investigation, Software. **Aman Ullah:** Writing – review & editing, Conceptualization. **Cobbinah B. Mawuli:** Writing – review & editing. **Qinli Yang:** Writing – review & editing, Conceptualization, Methodology, Supervision. **Junming Shao:** Writing – review & editing, Conceptualization, Methodology, Supervision.

Data availability

Data will be made available on request.

Acknowledgments

This work is supported by the Fundamental Research Funds for the Central Universities, China (ZYGX2019Z014), Sichuan Key Research Program (22ZDYF3388), National Natural Science Foundation of China (61976044, 52079026), Fok Ying-Tong Education Foundation for Young Teachers in the Higher Education Institutions of China (161062), National Key Research and Development Program, China (2016YFB0502300) and Sichuan Science and Technology Program, China (2022YFG0260).

References

- Aguiar, G., & Cano, A. (2023). An active learning budget-based oversampling approach for partially labeled multi-class imbalanced data streams. In J. Hong, M. Lanperne, J. W. Park, T. Cerný, & H. Shahriar (Eds.), *Proceedings of the 38th ACM/SIGAPP symposium on applied computing* (pp. 382–389). ACM.
- Al-Khateeb, T., Masud, M. M., Al-Naami, K. M., Seker, S. E., Mustafa, A. M., Khan, L., et al. (2016). Recurring and novel class detection using class-based ensemble for evolving data stream. *IEEE Transactions on Knowledge and Data Engineering*, 28(10), 2752–2764.
- Angelov, P. P., Gu, X., & Príncipe, J. C. (2018). A generalized methodology for data analysis. *IEEE Transactions on Systems, Man and Cybernetics*, 48(10), 2981–2993.
- Ashfahani, A., Pratama, M., Lughofer, E., & Ong, Y. S. (2020). DEVDAN: Deep evolving denoising autoencoder. *Neurocomputing*, 390, 297–314.
- Bifet, A., Holmes, G., Kirkby, R., & Pfahringer, B. (2010). MOA: Massive online analysis. *Journal of Machine Learning Research*, 11, 1601–1604.
- Bouguelia, M., Nowaczyk, S., & Payberah, A. H. (2018). An adaptive algorithm for anomaly and novelty detection in evolving data streams. *Data Mining and Knowledge Discovery*, 32(6), 1597–1633.
- Brzezinski, D., & Stefanowski, J. (2014). Reacting to different types of concept drift: The accuracy updated ensemble algorithm. *IEEE Transactions on Neural Networks and Learning Systems*, 25(1), 81–94.
- Cai, X. Q., Zhao, P., Ting, K. M., Mu, X., & Jiang, Y. (2019). Nearest neighbor ensembles: An effective method for difficult problems in streaming classification with emerging new classes. In *2019 IEEE international conference on data mining* (pp. 970–975). IEEE.
- Cano, A., & Krawczyk, B. (2020). Kappa updated ensemble for drifting data stream mining. *Machine Learning*, 109(1), 175–218.
- Cano, A., & Krawczyk, B. (2022). ROSE: robust online self-adjusting ensemble for continual learning on imbalanced drifting data streams. *Machine Learning*, 111(7), 2561–2599.
- de Faria, E. R., Ponce de Leon Ferreira Carvalho, A. C., & Gama, J. (2016). MINAS: multiclass learning algorithm for novelty detection in data streams. *Data Mining and Knowledge Discovery*, 30(3), 640–680.
- Din, S. U., Kumar, J., Shao, J., Mawuli, C. B., & Ndiaye, W. D. (2022). Learning high-dimensional evolving data streams with limited labels. *IEEE Transactions on Cybernetics*, 52(11), 11373–11384.
- Din, S. U., & Shao, J. (2020). Exploiting evolving micro-clusters for data stream classification with emerging class detection. *Information Sciences*, 507, 404–420.
- Din, S. U., Shao, J., Kumar, J., Ali, W., Liu, J., & Ye, Y. (2020). Online reliable semi-supervised learning on evolving data streams. *Information Sciences*, 525, 153–171.
- Din, S. U., Shao, J., Kumar, J., Mawuli, C. B., Mahmud, S. H., Zhang, W., et al. (2021). Data stream classification with novel class detection: A review, comparison and challenges. *Knowledge and Information Systems*, 63, 2231–2276.
- Fahy, C., Yang, S., & Gongora, M. (2022). Scarcity of labels in non-stationary data streams: A survey. *ACM Computing Surveys*, 55(2).
- Fahy, C., Yang, S., & Gongora, M. (2023). Classification in dynamic data streams with a scarcity of labels. *IEEE Transactions on Knowledge and Data Engineering*, 35(4), 3512–3524. <http://dx.doi.org/10.1109/TKDE.2021.3135755>.
- Ferreira, R. S., Zimbrão, G., & Alvim, L. G. (2019). Amanda: Semi-supervised density-based adaptive model for non-stationary data with extreme verification latency. *Information Sciences*, 488, 219–237.
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4), 1–37.
- Gao, Y., Chandra, S., Li, Y., Khan, L., & Bhavani, T. (2022). SACCOS: A semi-supervised framework for emerging class detection and concept drift adaption over data streams. *IEEE Transactions on Knowledge and Data Engineering*, 34(3), 1416–1426.
- Gomes, H. M., Bifet, A., Read, J., Barddal, J. P., Enembreck, F., Pfahringer, B., et al. (2017). Adaptive random forests for evolving data stream classification. *Machine Learning*, 106(9–10), 1469–1495.
- Gomes, H. M., Read, J., & Bifet, A. (2019). Streaming random patches for evolving data stream classification. In J. Wang, K. Shim, & X. Wu (Eds.), *2019 IEEE international conference on data mining* (pp. 240–249). IEEE.
- Gomes, H. M., Read, J., Bifet, A., Barddal, J. P., & Gama, J. (2019). Machine learning for streaming data: state of the art, challenges, and opportunities. *ACM SIGKDD Explorations Newsletter*, 21(2), 6–22.
- Gu, X. (2022). An explainable semi-supervised self-organizing fuzzy inference system for streaming data classification. *Information Sciences*, 583, 364–385.
- Gu, X. (2023). A dual-model semi-supervised self-organizing fuzzy inference system for data stream classification. *Applied Soft Computing*, 136, Article 110053.
- Gu, X., Angelov, P. P., & Príncipe, J. C. (2018). A method for autonomous data partitioning. *Information Sciences*, 460–461, 65–82.
- Guo, H., Xia, H., Li, H., & Wang, W. (2023). Concept evolution detection based on noise reduction soft boundary. *Information Sciences*, 628, 391–408.
- Haque, A., Khan, L., & Baron, M. (2016). SAND: Semi-supervised adaptive novel class detection and classification over data stream. *Proceedings of the AAAI Conference on artificial intelligence*, vol. 30, no. 1.
- Hasan, M., Orgun, M. A., & Schwitter, R. (2019). Real-time event detection from the Twitter data stream using the TwitterNews+ framework. *Information Processing and Management*, 56(3), 1146–1165.
- Hosseini, M. J., Gholipour, A., & Beigy, H. (2016). An ensemble of cluster-based classifiers for semi-supervised classification of non-stationary data streams. *Knowledge and Information Systems*, 46(3), 567–597.
- Khezri, S., Tanha, J., Ahmadi, A., & Sharifi, A. (2020). STDS: self-training data streams for mining limited labeled data in non-stationary environment. *Applied Intelligence*, 50(5), 1448–1467.
- Komorniczak, J., Zyblewski, P., & Ksieniewicz, P. (2022). Statistical drift detection ensemble for batch processing of data streams. *Knowledge-Based Systems*, 252, Article 109380.
- Korycki, L., Cano, A., & Krawczyk, B. (2019). Active learning with abstaining classifiers for imbalanced drifting data streams. In C. K. Baru, J. Huan, L. Khan, X. Hu, R. Ak, Y. Tian, & et al. (Eds.), *2019 IEEE international conference on big data* (pp. 2334–2343). IEEE.
- Kranjc, J., Smailovic, J., Podpecan, V., Grcar, M., Znidarsic, M., & Lavrac, N. (2015). Active learning for sentiment analysis on data streams: Methodology and workflow implementation in the CloudFlows platform. *Information Processing and Management*, 51(2), 187–203.
- Krawczyk, B., & Cano, A. (2019). Adaptive ensemble active learning for drifting data stream mining. In S. Kraus (Ed.), *Proceedings of the Twenty-eighth international joint conference on artificial intelligence* (pp. 2763–2771). ijcai.org.
- Liao, G., Zhang, P., Yin, H., Deng, X., Li, Y., Zhou, H., et al. (2023). A novel semi-supervised classification approach for evolving data streams. *Expert Systems with Applications*, 215, Article 119273.
- Losing, V., Hammer, B., & Wersing, H. (2017). Self-adjusting memory: How to deal with diverse drift types. In *Proceedings of the twenty-sixth international joint conference on artificial intelligence* (pp. 4899–4903).
- Losing, V., Hammer, B., & Wersing, H. (2018). Tackling heterogeneous concept drift with the self-adjusting memory (SAM). *Knowledge and Information Systems*, 54(1), 171–201.
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2019). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363.
- Masud, M., Gao, J., Khan, L., Han, J., & Thuraishingham, B. M. (2010). Classification and novel class detection in concept-drifting data streams under time constraints. *IEEE Transactions on knowledge and data engineering*, 23(6), 859–874.
- Mohamad, S., Sayed Mouchaweh, M., & Bouchachia, A. (2018). Active learning for classifying data streams with unknown number of classes. *Neural Networks*, 98, 1–15.

- Mu, X., Ting, K. M., & Zhou, Z.-H. (2017). Classification under streaming emerging new classes: A solution using completely-random trees. *IEEE Transactions on Knowledge and Data Engineering*, 29(8), 1605–1618.
- Mu, X., Zhu, F., Du, J., Lim, E., & Zhou, Z. (2017). Streaming classification with emerging new class by class matrix sketching. In *Proceedings of the thirty-first AAAI conference on artificial intelligence* (pp. 2373–2379). AAAI Press.
- Narasimhamurthy, A., & Kuncheva, L. I. (2007). A framework for generating data to simulate changing environments. In *Proceedings of the 25th conference on proceedings of the 25th IASTED international multi-conference: Artificial intelligence and applications* (pp. 384–389).
- Nguyen, M. L., Gomes, H. M., & Bifet, A. (2019). Semi-supervised learning over streaming data using MOA. In C. K. Baru, J. Huan, L. Khan, X. Hu, R. Ak, Y. Tian, & et al. (Eds.), *2019 IEEE international conference on big data* (pp. 553–562). IEEE.
- Parmar, J., Chouhan, S. S., & Raychoudhury, V. (2023). A machine learning based framework to identify unseen classes in open-world text classification. *Information Processing & Management*, 60(2), Article 103214.
- Pham, T., Kottke, D., Kreml, G., & Sick, B. (2022). Stream-based active learning for sliding windows under the influence of verification latency. *Machine Learning*, 111(6), 2011–2036.
- Razavi-Far, R., Hallaji, E., Saif, M., & Ditzler, G. (2019). A novelty detector and extreme verification latency model for nonstationary environments. *IEEE Transactions on Industrial Electronics*, 66(1), 561–570.
- Shao, J., Huang, C., Yang, Q., & Luo, G. (2016). Reliable semi-supervised learning. In F. Bonchi, J. Domingo-Ferrer, R. Baeza-Yates, Z. Zhou, & X. Wu (Eds.), *IEEE 16th International Conference on Data Mining* (pp. 1197–1202). IEEE Computer Society.
- Shao, J., Huang, F., Yang, Q., & Luo, G. (2018). Robust prototype-based learning on data streams. *IEEE Transactions on Knowledge and Data Engineering*, 30(5), 978–991.
- Wagner, T., Guha, S., Kasiviswanathan, S. P., & Mishra, N. (2018). Semi-supervised learning on data streams via temporal label propagation. In J. G. Dy, & A. Krause (Eds.), *Proceedings of machine learning research: vol. 80, Proceedings of the 35th International Conference on Machine Learning* (pp. 5082–5091). PMLR.
- Wang, Y., & Li, T. (2018). Improving semi-supervised co-forest algorithm in evolving data streams. *Applied Intelligence*, 48(10), 3248–3262.
- Yin, C., Chen, S., & Yin, Z. (2023). Clustering-based active learning classification towards data stream. *ACM Transactions on Intelligent Systems and Technology*.
- ZareMoaddi, P., Kamali Siahroudi, S., & Beigy, H. (2019). Concept-evolution detection in non-stationary data streams: a fuzzy clustering approach. *Knowledge and Information Systems*, 60, 1329–1352.
- Zheng, X., Li, P., Hu, X., & Yu, K. (2021). Semi-supervised classification on data streams with recurring concept drift and concept evolution. *Knowledge-Based Systems*, 215, Article 106749.
- Zhu, L., Pang, S., Sarrafzadeh, A., Ban, T., & Inoue, D. (2016). Incremental and decremental max-flow for online semi-supervised learning. *IEEE Transactions on Knowledge and Data Engineering*, 28(8), 2115–2127.